République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Sousse

Institut Supérieur des Sciences
Appliquées et de Technologie
de Sousse

DEPARTEMENT INFORMATIQUE

RAPPORT DE PROJET DE FIN D'ETUDES

En vue de l'obtention du:
Diplôme National d'Ingénieur Informatique
Option: Génie Logiciel-Architecture Logicielle

Développement d'une solution "Force de Vente"

Elaboré par :

Mohamed Ben Halima

Soutenu le 06/07/2022 devant le jury :

Président :	**Mme Ameni Youssfi**	**ISSAT de Sousse**
Examinateur :	**Mme Emna Ammar**	**ISSAT de Sousse**
Encadr¨æ¹ :	**M. Sami Achour**	**ISSAT de Sousse**
Encadrant Industriel :	**M. Mourad Jemail**	**Inetum Holding**

Année Universitaire : 2021 / 2022

Code Sujet: FI-GL22-007

Dédicaces

Toutes les lettres ne sauraient trouver les mots qu'il faut. Tous les mots ne sauraient exprimer la gratitude, l'amour, le respect, la reconnaissance. Aussi, c'est tout simplement que je dédie ce projet de fin d'études...

À mes chers parents Samir et Sonia

Qui n'ont pas cessé de m'aider, de me soutenir et de m'encourager. En espérant qu'ils trouvent ici le résultat de nombreuses années de sacrifice et de privation pour m'aider à avancer dans la vie.

À mon adorable sœur Hadile

Les bijoux de la famille à qui je donnerai tout, je te souhaite un avenir plein de bonheur, de réussite et de sérénité.

À tous mes amis

Auprès desquels j'ai passé d'agréables moments pendant ces dernières années, je leur souhaite tout le succès et un avenir brillant.

Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à mon professeur et encadrant Monsieur Sami Achour pour son suivi et pour son énorme soutien qu'il n'a cessé de me prodiguer tout au long de la période du projet.

Je tiens à remercier également mon encadrant professionnel Monsieur Mourad Jemail de la société Inetum pour leurs précieux conseils, informations et motivations qu'il m'a fournis et qu'il m'a permis de tenir jusqu'au bout pour réussir ce projet.

J'aimerais remercier tous mes enseignants d'avoir partager avec moi leur passion pour l'enseignement. J'ai grandement apprécié leurs appuis, implications et expériences tout au long de ma formation.

J'adressais aussi mes vifs remerciements à tous les membres de la société Inetum tels que Mme. Meriem Guesmi, M. Tarek Taamali, Mme. Ahlem Ben Guiza et Mme Oumaima Boukhris auprès desquels, j'ai trouvé l'accueil chaleureux, l'aide et l'assistance dont j'ai besoin, un environnement dans lequel nous avons pu nous adapter facilement.

Table des matières

Introduction Générale **1**

1 Présentation générale du projet **3**

1.1 Présentation de l'organisme d'accueil 3

1.1.1 Domaine d'activité 4

1.1.2 Organigramme de la société 5

1.2 Présentation du projet 5

1.2.1 Cadre général du projet 5

1.2.2 Analyse de l'existant 5

1.2.3 Critique de l'existant 9

1.2.4 Solution envisagée 9

1.2.5 Objectifs 10

1.3 Processus de développement 10

1.3.1 La méthode agile 11

1.3.2 La Méthodologie Scrum 11

1.3.3 Contrôle de version 14

2 Spécification des besoins **16**

2.1 Identification des acteurs 16

2.2 Besoins Fonctionnels 17

2.2.1 Application mobile 17

2.2.2 Application web 19

2.3 Besoins non fonctionnels 20

2.4 Diagrammes de cas d'utilisation 21

2.4.1 Diagramme de cas d'utilisation générale 22

2.4.2 Diagramme de cas d'utilisation de l'agent de point de vente . 23
2.4.3 Diagramme de cas d'utilisation de l'agent commercial 23
2.4.4 Diagramme de cas d'utilisation de l'Admin 24
2.5 Planification des tâches 25

3 Conception **28**
3.1 Introduction 28
3.2 Architecture 3-tiers 28
3.3 Patrons de conception 29
3.3.1 Patron de conception clean architecture 30
3.3.2 Patron de conception MVC 32
3.4 Conception détaillée 33
3.4.1 Diagrammes de séquences 36
3.4.2 Architecture logicielle générale 40

4 Réalisation **42**
4.1 Introduction 42
4.2 Environnement de développement 42
4.2.1 Technologies utilisées 42
4.2.2 Langages de programmation 44
4.2.3 Outils 45
4.3 Diagramme de déploiement 50
4.4 Travail réalisé 50
4.4.1 Continuous Integration/Continuous Delivery | CI/CD 50
4.4.2 Inetgration d'une solution SSO 51
4.4.3 Maquettage 52
4.4.4 Interface choix de rôle 53
4.4.5 Interface Authentification 53
4.4.6 Interface d'accueil 54
4.4.7 Interface catalogue produits 55
4.4.8 Interface de détail de produit et commande 56
4.4.9 Interface panier 57
4.4.10 Interface commande envoyer 58

4.4.11 Interface profile 59
4.4.12 Interface ajouter objectif 60
4.4.13 Interface menu barre 62
4.4.14 Interface Factue 62
4.4.15 Interface réclamation 63
4.4.16 Interface ajouter réclamation 64
4.4.17 Interface historique de commande 65

Conclusion et prespectives **67**

Table des figures

1.1 Inetum Tunisie en Chiffre 4
1.2 Organigramme de la société Inetum 5
1.3 Page d'accueil de l'application VENDY 6
1.4 l'application mobile Yuto [26] 7
1.5 Tableau de bord de l'application Sellsy [24] 8
1.6 Processus de scrum 14
1.7 Tableau de bord de l'application ClickUp 14
1.8 Fréquence des commits Github 15

2.1 Diagramme du cas d'utilisation générale 22
2.2 Diagramme du cas d'utilisation de l'agent de point de vente 23
2.3 Diagramme du cas d'utilisation de l'agent commercial 24
2.4 Diagramme du cas d'utilisation de l'admin de la solution 25

3.1 Architecture 3-tiers [1] 29
3.2 Clean architecture d'après Uncle Bob [3] 31
3.3 Extrait de l'architecture de l'application mobile "Inetum Force de vente" 31
3.4 Principe du patron de conception MVC [19] 32
3.5 Extrait de l'architecture du backend de l'application "Inetum Force de vente" 33
3.6 Diagramme de classes des entités 35
3.7 DS du cas d'utilisation «Authentification» 37
3.8 DS du cas d'utilisation «Gérer réclamation» 38
3.9 DS du cas d'utilisation «Gérer commandes» 40
3.10 Architecture logicielle générale 41

4.1 Diagramme de déploiement 50
4.2 plateform Heroku 51
4.3 Interface keycloak 52
4.4 Maquette graphique de l'application Inetum Force de vente 52
4.5 Interface choix de rôle 53
4.6 Interface d'authentification 54
4.7 Interface d'accueil 55
4.8 Interface catalogue produits 56
4.9 Interface détails du produit 57
4.10 Interface panier 58
4.11 Interface commande envoyée 59
4.12 Interface profile 60
4.13 Interface 'ajouter objectif' 61
4.14 Interface menu barre 62
4.15 Interfaces facture 63
4.16 Interfaces réclamation 64
4.17 Interfaces ajouter réclamation 65
4.18 Interfaces historique de commande 66

Liste des tableaux

2.1 Tableau d'identification des acteurs 17
2.2 Product backlog 27

3.1 Tableau descriptif du diagramme de classes des entités 34

Liste des Acronymes

- **API :** Application Programming Interface
- **DS :** Diagramme de Séquence
- **ERP :** Enterprise Resource Planning
- **HTTP :** HyperText Transfer Protocol
- **IHM :** Interface Homme/Machine
- **MVC :** Model-View-Controller
- **UML :** Unified Modeling Language
- **SSO :** Single sign-on
- **PV :** Point de vente
- **CI/CD :** Continuous Integration/Continuous Delivery

Introduction Générale

La force de vente regroupe l'ensemble du personnel d'une entreprise censée participer à la vente de produits ou services. Tous les employés et commerciaux indépendants qui permettent d'augmenter le chiffre de l'entreprise par la commercialisation d'articles ou services y sont intégrés.

A cet égard, adoptez une solution de force de vente est l'un parmit les meilleurs choix pour dynamiser les activité commerciale, tel qu'elle ameliore la communication entre les clients et l'entreprise, optimise la gestion de stock, assure une bonne interraction entre les employé de l'entreprise et augmente le taux d'engagement de clients.

C'est pour cela que nous avons pris l'initiative de concevoir une application qui a pour objectif de faciliter la communication entre les differents acteurs du concept de force de vente.

Le présent rapport a pour objectif de donner une idée claire sur le travail réalisé et sur les étapes de réalisation. Il comporte quatre chapitres organisés comme suit :

- Le premier chapitre présente le contexte du projet et la problématique soulevée
- Le second chapitre sera dedié à la spécification des différents besoins de l'application
- Le troisième chapitre présente les aspects conceptuels du projet

- Le quatrième chapitre traite tous les détails de la réalisation ainsi qu'un aperçu sur les importants résultats obtenus.

Nous finissons par une conclusion générale qui résumera le travail et contiendra des perspectives d'avenir.

Chapitre 1

Présentation générale du projet

Introduction

Nous commençons par ce chapitre qui s'intéresse à mettre notre projet dans son contexte général. Ensuite nous précisons la méthode adoptée et le cycle de vie de notre projet en mettant en phase les motivations qui nous ont encouragés lors de sa réalisation.

1.1 Présentation de l'organisme d'accueil

Inetum est une ESN agile, une société de services et de solutions digitales, et un groupe international qui aide les entreprises et institutions à tirer le meilleur du digital flow. Avec son profil de multi-spécialiste, Inetum met au service de ses clients une combinaison unique de proximité, d'organisation sectorielle et de solutions de qualité industrielle.





FIGURE 1.1 – Inetum Tunisie en Chiffre

1.1.1 Domaine d'activité

Les activités de la société Inetum sont les suivantes :

- Conseil et services
- Transformation digitale :
  - Développement spécifiques
  - Portails et mobile / UX
  - IOT / Industrie 4.0
  - Migrations applicatives
- Business solutions
  - Sages
  - SAP
  - Microsoft
  - SIRH
- BI et Data science

1.1.2 Organigramme de la société

La figure 1.2 présente l'organigramme de la société Inetum Tunisie.

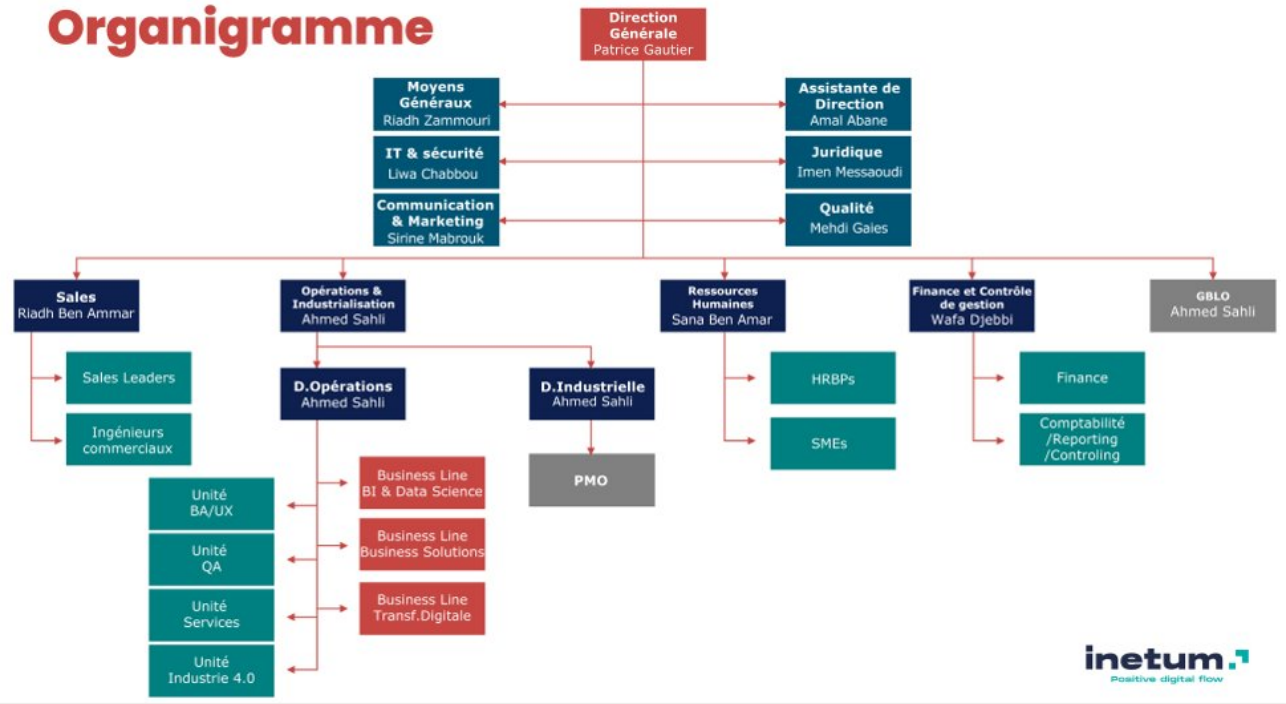


FIGURE 1.2 – Organigramme de la société Inetum

1.2 Présentation du projet

1.2.1 Cadre général du projet

Le présent projet intitulé "INETUM FORCE DE VENTE" est réalisé dans le cadre d'un stage de fin d'études afin d'obtenir le diplôme national d'ingénieur à l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse.

1.2.2 Analyse de l'existant

Dans le domaine des applications force de vente, plusieurs produits et services sont proposés. Cependant, nous choisissons de parler sur les produits et les services qui nous sommes rapprochés du besoin que nous voulons combler. Après une étude approfondie, nous avons pu étudier trois solutions existantes qui ciblent à peu près notre problémtique.

L'application VENDY

VENDY (cf. figure 1.3) est une application interne de la société Chahia Tunisie, utilisé pour garantir une bonne communication entre le siège de l'entreprise et les commerciaux.

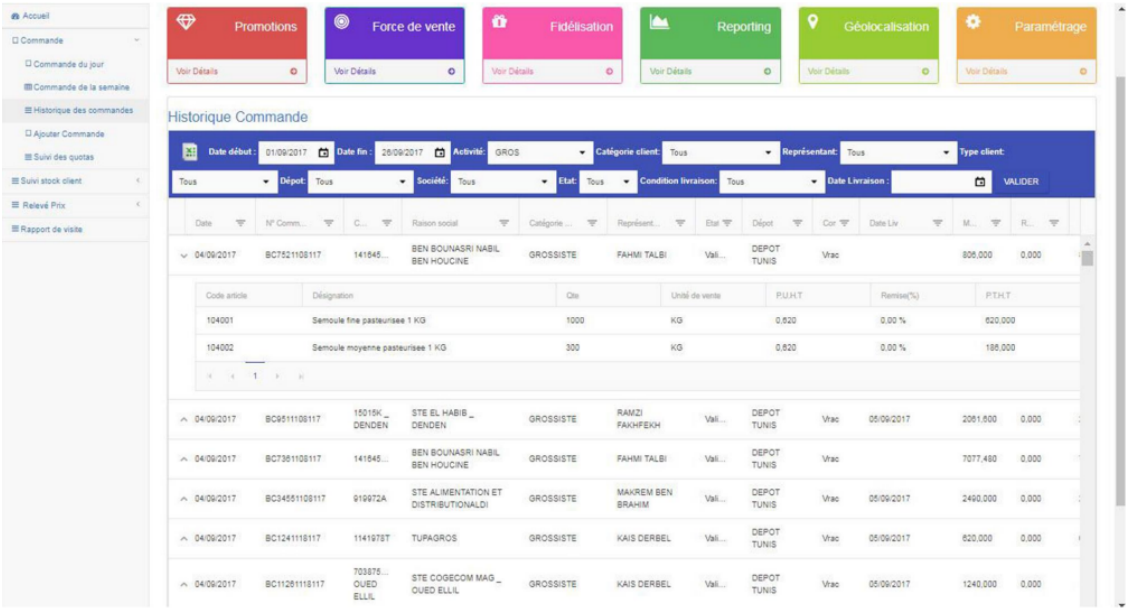


FIGURE 1.3 – Page d'accueil de l'application VENDY

- **Parmis les avantages nous citons :**
  - Réponds à la majorité des fonctionnalités de force de vente.
  - Fonctionne sur tous les navigateurs web.
  - Intégré avec l'ERP de l'entreprise.
- **Parmis les inconvénients nous citons :**
  - Une expérience utilisateur complexe et ennuyante
  - Mauvaise ergonomie, non compatible avec les petits appareils.
  - Application très lente.
  - Absence d'une version mobile friendly.
  - Absence de notifications.

L'application YUTO

l'application Yuto (cf. figure 1.4) est un compagnon polyvalent pour les commerciaux. Non seulement elle enregistre tous les échanges avec les clients et prospects, mais elle facilite la création de devis, permet de disposer d'un catalogue actualisé en temps réel. Ses nombreuses fonctionnalités visent notamment à automatiser les tâches les plus chrono-phages pour les vendeurs, du suivi de clientèle à la vérification des objectifs. De plus, il est compatible avec la plupart des ERP[1].

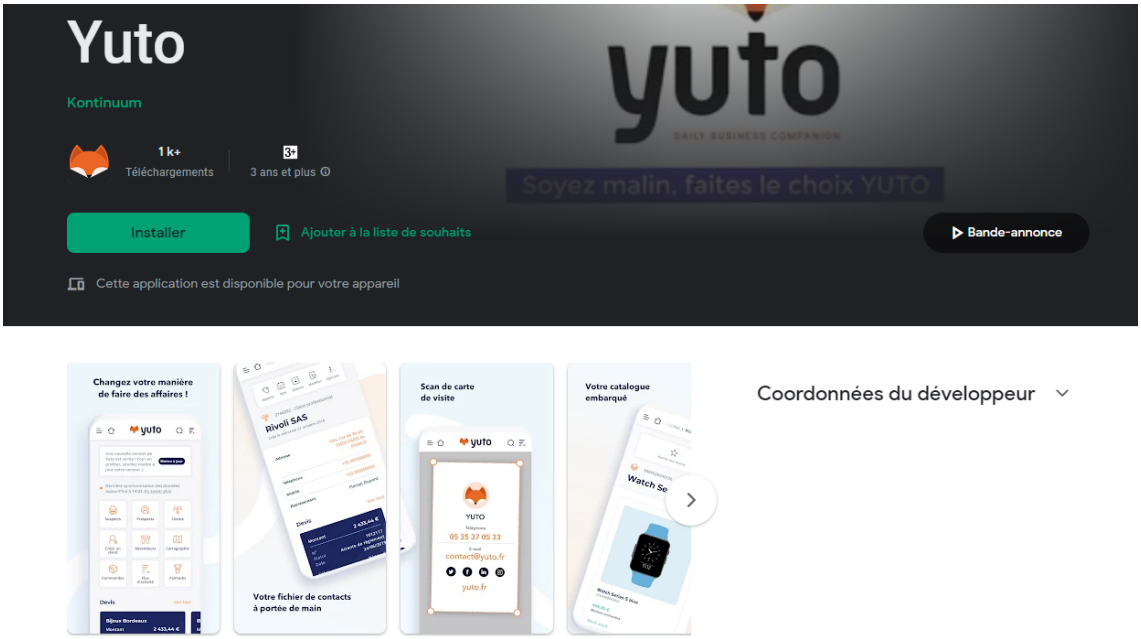


FIGURE 1.4 – l'application mobile Yuto [26]

- **Parmis les avantages nous citons :**
  - Réponds à la majorité des fonctionnalités de force de vente.
  - Une experience utilisateur claire et facile.
  - Intégre la majorité des ERP sur le marché.
- **Parmis les inconvénients nous citons :**
  - La licence est trop coûteuse.

L'Application Sellsy

L'application Sellsy (cf. figure 1.5) propose une solution modulable à toutes les entreprises, en fonction de leurs besoins. En effet, il est possible de construire son

offre personnalisée, incluant au choix un module CRM, un module de facturation et un module de fidélisation. Côté facturation, Sellsy facilite la création de devis et de factures. Elle facilite le paiement en mettant à disposition du client signatures électroniques et solution de paiement en ligne. De plus, elle permet d'automatiser la facturation récurrente et offre une analyse par produit, ou par dossier. Côté fidélisation, Sellsy offre une solution complète pour mieux segmenter la clientèle et mettre en place des propositions commerciales ciblées.

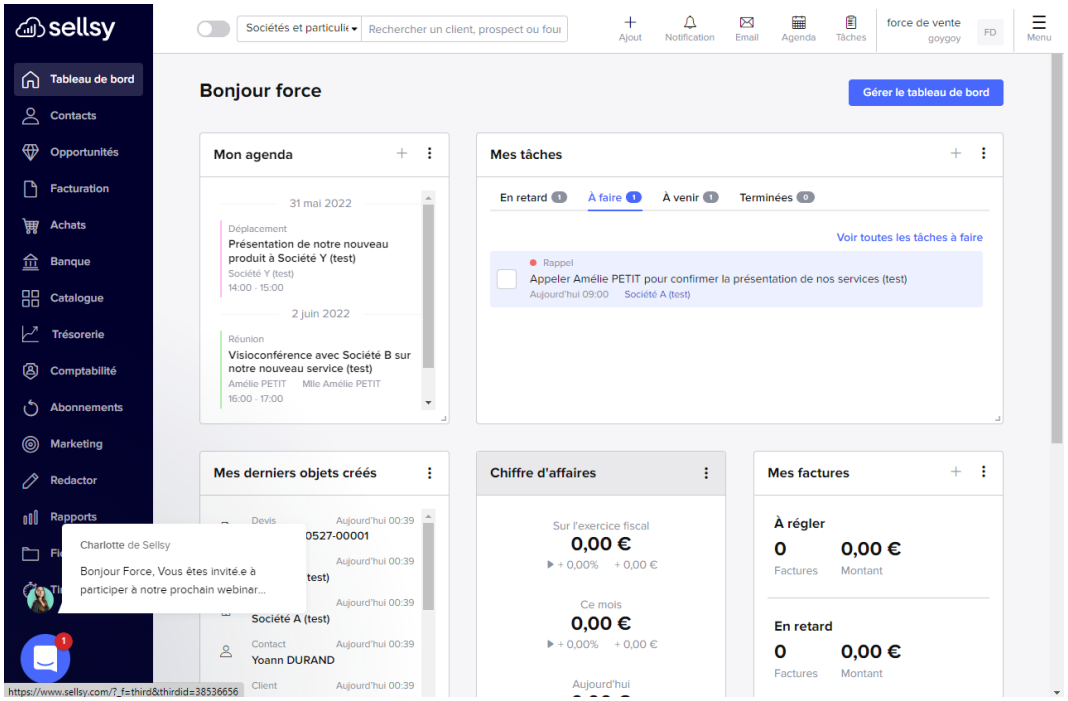


FIGURE 1.5 – Tableau de bord de l'application Sellsy [24]

- **Parmis les avantages nous citons :**
  - Réponds à la majorité des fonctionnalités de force de vente.
  - Fonctionne sur tous les navigateurs web.
- **Parmis les inconvénients nous citons :**
  - Une expérience utilisateur complexe et ennuyante.
  - Non compatible avec les ERP du marché.
  - N'est pas responsive pour les petits appareil.

1.2.3 Critique de l'existant

Même si les solutions que nous avons mentionnées répondent aux besoins des entreprises et leur permettent de digitaliser le concept de force de vente, nous envisageons d'ajouter nos clés d'amélioration étant donné que ces derniers comportent des incohérences et des points faibles notamment :

- Lenteur du chargement.
- Absence de notifications.
- Présentation encombrée des données.

1.2.4 Solution envisagée

Après avoir étudié les défauts des solutions actuelles, notre proposition dans ce projet de fin d'études est de réaliser une plateforme web et mobile, avec laquelle, les salariés et les commerciaux indépendants peuvent communiquer et échanger les informations facilement et en temps réel, ce que aide l'entreprise à garantir une meilleur productivité et une centralisation de données. Notre application intitulé INETUM FORCE DE VENTE est caractérisé par les fonctionnalités suivantes :

- Un module d'actualité, ce qui donne aux commerciaux la possibilité de consulté les actualités de l'entreprise.
- Une rubrique de promotion, ou les utilisateur peuvent consulter les nouvelles promotion proposé par l'entreprise.
- Un module de gestion des commandes, qui permet aux utilisateurs de passer, consulter, supprimer et modifier des commandes en temps réel à partir d'interfaces graphiques simples et faciles.
- Un module de gestion des factures, qui permet aux agents de points de vente de consulter leurs factures et leur statut.
- Un module de gestion de réclamation, qui permet aux utilisateurs de consulter et passer des réclamations en ligne.

- Un module de gestion des objectifs, qui permet aux utilisateurs de créer des objectifs de ventes mensuelles.
- Une carte géographique, qui permet aux commerciaux de visualiser et d'ajouter le points de vente partenaires et concurent sur la carte.
- Un module connecteur entre notre application et l'ERP de l'entreprise qui permet la communication entre les deux systems.
- Un système de notification, qui garantit une communication interactive entre les différents acteurs de l'application.

1.2.5 Objectifs

Sous la supervision de notre tuteur, notre mission était de développer une plateforme web et mobile avec sa partie backend qui a pour objectif de satisfaire les principes de concept de force de vente mentionnés ci-aprés :

- Accueillir des clients potentiels ou actuels dans le but d'assurer la promotion, la vente et l'après vente des produits ou services d'une entreprise.
- Adhérer à la politique et la stratégie générale de l'entreprise.
- Assurer une bonne communication entre toutes les personnes impliquées dans le processus de vente de produits ou de services de l'entreprise.
- Faciliter le suivi et l'interaction entre les clients d'entreprise, ses représentants et l'administration du siège.
- Offrir une bonne experience utilisateur pour les différents acteurs de l'application.
- Communiquer avec l'ERP de l'entreprise.

1.3 Processus de développement

Les projets informatiques sont de plus en plus sophistiqués en raison des variations de besoins et des spécifications qui sont très importants lors de l'élaboration

du projet. C'est pourquoi les informaticiens ont réinventé les méthodes de gestion de projet. Le choix d'une méthodologie appropriée pour entreprendre un projet est une étape cruciale qui détermine son succès. Le respect d'une méthode bien définie permet à un projet de mieux estimer, livrer des systèmes stables, tenir le client informé, créer une compréhension claire de la tâche à venir, et identifier les pièges plus tôt pour ajustements.
Dans notre entreprise Inetum, nous avons constaté que méthodologie Agile SCRUM, est la méthodologie la plus appropriée pour notre projet. elle a pour objectif de décrir comment notre solution devrait assurer la préparation, la planification, la surveillance et le contrôle des activités nécessaire pour terminer les tâches requises. Ce choix est basé sur la nécessité de livrer les diffusions régulières, l'exigence de rétroaction périodique et les changements dans les exigences du projet au fil du temps.

1.3.1 La méthode agile

Les méthodes agiles sont un ensembles de pratiques pour la gestion de projets informatiques. Ces méthodes viser à impliquer de plus en plus le client afin d'offrir au projet une grande réactivité à leurs demandes et exigences . Le concept essentiel est de fixer un objectif à court terme qui correspond aux fonctionnalités à développer. Le partie des exigences choisies pour chaque objectif est appelée itération. Cette phase est répété jusqu'à l'obtention des différentes exigences des spécifications fournies par le client. Le défi des méthodologies de développement reste de répondre aux délais de livraison et aux besoins de tout projet informatique.

1.3.2 La Méthodologie Scrum

Scrum est la méthodologie la plus utilisée parmi les méthodes Agile existantes. La terme Scrum apparu pour la première fois en 1986 dans une publication qui décrit une approche nouvelle et plus souple du développement de nouveaux produits. Ils ont ensuite comparé cette nouvelle méthode d'union de rugby, le principe de base étant que l'équipe se déplace ensemble et est toujours prêt à réorienter le projet au fur et à mesure qu'il progresse, comme un ballon de rugby qui doit passer d'une main à l'autre jusqu'à ce qu'un essai soit marqué.

Dans ce contexte, un Sprint est une itération. Il s'agit d'une période de 2 à 4 semaines maximum au cours de laquelle une version complète et utilisable du produit est produite. Un nouveau sprint commence à la fin du précédent. Chaque sprint a un but et un liste des caractéristiques à réaliser. Dans chaque sprint, il y a la planification de sprint, standup quotidien, sprint revue et sprint rétrospective. Il y a des tableaux de travail et de l'épuisement graphiques pour suivre la progression du sprint ainsi que pour recevoir des retours incrémentiels.

Les rôles en scrum

- **Product Owner :** La vision du logiciel à construire est communiquée par le Product Owner. ce dernier se concentre non seulement sur le travail à terminer, mais aussi sur les exigences commerciales du marché.
  Le Product Owner interagit avec l'équipe ainsi qu'avec d'autres parties prenantes pour créer et gérer le backlog. Le rôle d'un Product Owner est de motiver l'équipe et l'aligner sur les objectifs et la vision du projet.
  Dans ce projet, le Product Owner est mon superviseur M. Mourad Jemail.

- **Scrum Master :** Le scrum Master est le responsable de l'organisation des réunions, traiter les défis et eviter les blocages. Il interagit avec le Product Owner pour s'assurer que le product backlog est prêt pour le prochain sprint.
  Dans ce projet, le Scrum Master est M. Tarek Taamali.

- **Scrum Team :** L'équipe Scrum peut être composée de 5 à 7 membres. Dans une équipe Scrum, il n'y a pas de rôles distincts en tant que programmeur, concepteur ou testeur, mais chacun a un ensemble de tâches qu'il accomplit ensemble.
  L'équipe Scrum planifie la quantité de travail qu'elle peut effectuer à chaque itération.
  Dans ce projet, l'équipe Scrum est composé de M. Mourad Jemail, M. Tarek Taamali, M. Mohamed Ben Halima, Mme Zeineb sfaxi et Mme Amani Bahlous.

Étapes du flux Scrum

- **Product backlog :** Le backlog produit comprend une liste de toutes les fonctionnalités souhaitées du produit. Le Product Owner et le Scrum Master hiérarchisent les éléments sur la base des user stories et des exigences. L'équipe de développement se réfère au backlog du produit pour terminer la tâche lors de chaque sprint.

- **Sprint planning :** Lors de la réunion de planification de sprint, le Product Owner fournit une liste des éléments hautement prioritaires du backlog. L'équipe choisit la tâche qui peuvent terminer pendant le sprint et transférer les tâches du backlog de produit vers le backlog de sprint.

- **Backlog refinement :** L'équipe et le Product Owner se rencontrent à la fin de chaque sprint pour préparer le backlog du sprint suivant. L'équipe divise les user stories sous forme des tâches raffiné et supprime toutes les user stories qui ne sont pas pertinentes. L'équipe accède également à la priorité des user story pour re-prioriser les tâches.

- **Daily Scrum :** Une réunion debout de 15 minutes connue sous le nom de Daily Scrum est organisée quotidiennement. Le membre de l'équipe discute des objectifs et des problèmes liés au développement. Le Daily Scrum a lieu tous les jours pendant le sprint pour garder l'équipe sur la bonne voie.

- **Sprint review meeting :** Une démonstration en direct est donnée à la fin de chaque sprint pour présenter le travail que l'équipe a réalisé.

- **Sprint retrospective meeting :** Cette réunion est organisée pour réfléchir au succès du processus Scrum et s'il ya des changements à apporter au prochain sprint. L'équipe discute les hauts et les bas du sprint précédent et de toutes les améliorations pour le prochain sprint.

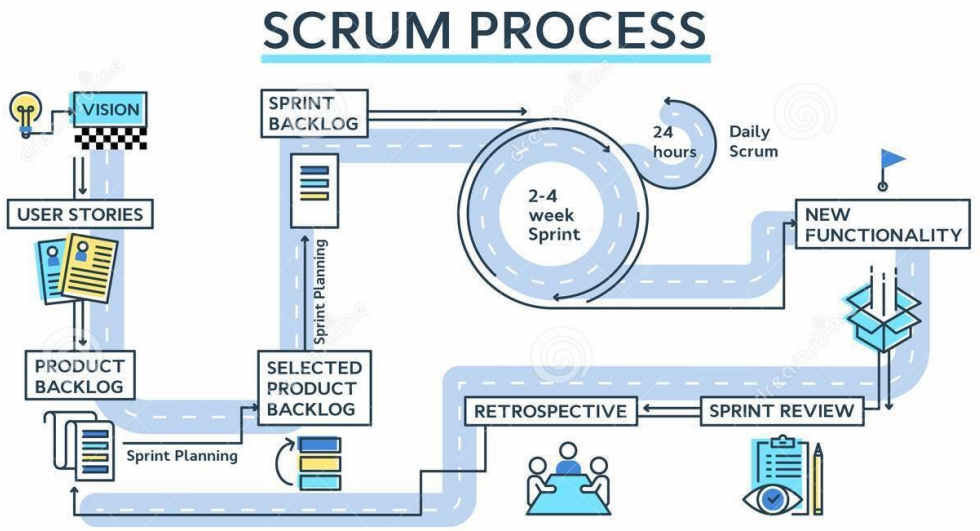


FIGURE 1.6 – Processus de scrum

Sur la base du flux Scrum décrit précédemment, chaque sprint a ses tickets de tâches qui sont insérés dans un outil en ligne que nous utilisons dans Inetum connu sous le nom de ClickUp, comme le montre la figure 1.7, le bloc 'Open' contient les tâches du sprint actuel provenant du backlog général , lorsqu'il travaille sur une tâche, le développeur déplace le ticket vers 'In progress', après il y a deux résultats, soit confirmer son comportement et lui donner le statut 'Done', soit le déplacer vers la section 'Review', en attendant une aide ou des ressources externes.

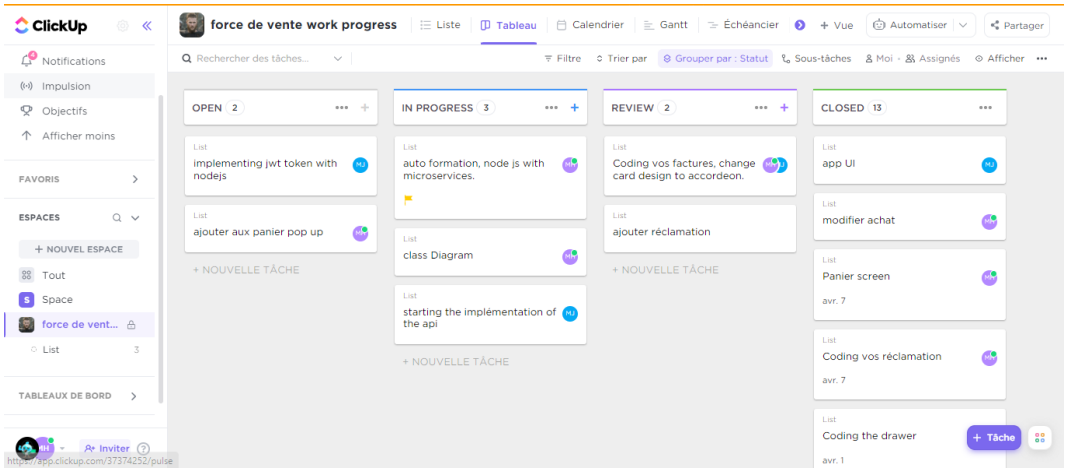


FIGURE 1.7 – Tableau de bord de l'application ClickUp

1.3.3 Contrôle de version

Il est impossible de décrire une gestion de projet basée sur la méthode agile sans mentionner le contrôle de version de code qui est essentiel à chaque équipe

travaillant sur le même projet.
Un système de contrôle de version ou VCS est un outil permettant de gérer plusieurs versions d'informations et de faciliter ensuite le travail collaboratif. Gérer manuellement les différentes versions des fichiers du projet est possible mais fastidieux. Les avantages d'un VCS sont multiples :

- Conservez l'historique du projet.
- Facilitez le travail collaboratif en aidant à détecter et à résoudre les conflits potentiels lors des modifications simultanées d'un fichier par différents utilisateurs.
- Revenir à une version précédente en cas d'erreur.
- Gérer plusieurs versions d'un même projet.

Dans notre projet, pour la gestion du code source, le suivi des problèmes de suivi et l'intégration continue, nous avons utilisé **Git** et la plateforme en ligne **Github** en sauvegardant l'historique de l'avancement de nos travaux.
La figure 1.8 suivante est un aperçu sur la fréquence de mes contributions personnelles :

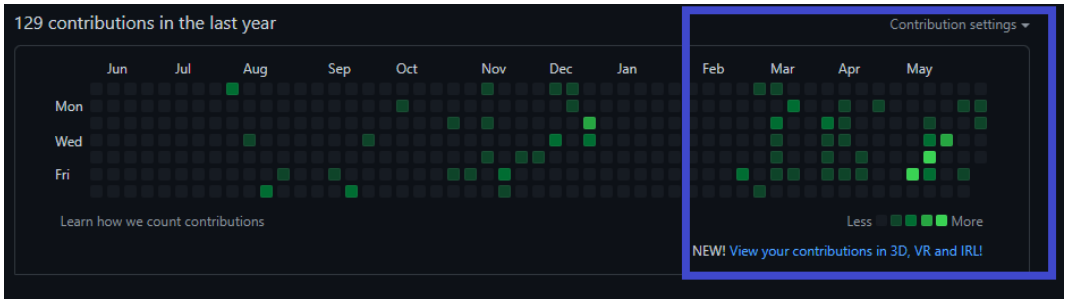


FIGURE 1.8 – Fréquence des commits Github

Conclusion

Nous avons détaillé le cadre général de notre solution à travers ce chapitre introductif. En présentant l'application que nous allons apprécier et développer, et les objectifs visés. Dans la suite nous allons entamer la phase de la conception de notre projet pour montrer les différentes fonctionnalités de l'application.

Chapitre 2

Spécification des besoins

Introduction

Continuons avec ce deuxième chapitre qui prend en charge de mettre en place la description de notre solution, pour mieux éclaircir les besoins des différents utilisateurs.
Nous déterminons les acteurs et leurs besoins fonctionnels et non fonctionnels puis nous terminons avec une description plus détaillée des besoins principaux en schématisant les différents cas d'utilisations sous forme de diagrammes UML.

2.1 Identification des acteurs

Les acteurs se présentent en tant qu'entités extérieures qui interagissent avec le système, ils peuvent être une personne, un matériel ou un logiciel.
Dans notre projet, les acteurs sont :

Acteurs	Description
Agent point de vente	Cet acteur est le propriétaire d'un point de vente ou il peut créer et gérer des actions(commander, réclamer...) sous la supervision du commercial et l'admin.
Commercial	C'est un acteur qui peut visualiser, valider ou refuser, les actions issue de l'agent de point de vente et avoir des informations sur les output de l'ERP.
Administrateur	C'est un acteur qui peut gérer les comptes et recevoir les actions de commercial et d'agents de point de vente.

TABLE 2.1 – Tableau d'identification des acteurs

2.2 Besoins Fonctionnels

La notion de force de vente a pour objectif de gagner et attacher les clients à l'entreprise le maximum possible dans le but de faire de la prospection, la vente et la collecte des informations, c'est pourquoi nous avons pensé à développer cette solution pour la digitaliser et répondre aux inspirations fonctionnelles des différents acteurs de la solution.
Nous décrivons par la suite les différents besoins fonctionnels des chaque acteur.

2.2.1 Application mobile

Cette application sera destinée pour l'agent de point de vente et l'agent commercial. Elle a pour objectif de faciliter et digitaliser la communication, la vente et l'achat entre eux et le siège de l'entreprise.

- **Un agent point de vente peut** :
  - S'authentifier.

    * Gérer les commandes :
        · Créer une commande.
        · Afficher les commandes et leurs détails.
        · Consulter le statut de la commande.
        · Consulter l'historique mensuel des commandes.
        · Envoyer les commandes
    * Gérer les réclamations :
        · Créer une réclamation.
        · Consulter l'état de la réclamation.
        · Consulter l'historique de réclamation.
    * Gérer les objectifs de vente.
        · créer un objectif .
        · Modifier un objectif.
        · Supprimer un objectif.
        · Consulter les objectifs.
    * Consulter les factures.
    * Consulter les promos et l'actualité de l'entreprise.
    * Consulter les notifications.
    * Consulter son profile.
    * Consulter le catalogue produits.

- **Un agent commercial peut** :

- S'authentifier.

  – Gérer les commandes :

    * Ajouter des commandes par point de vente.
    * Valider/Refuser les commandes crée par les agents de point de vente.
    * modifier les commandes réalisées par l'agent de point de vente en cas de besoin.

  – Gérer les points de vente :

        * Créer un compte pour chaque 'point de vente'.
        * Modifier un compte 'point de vente'.
        * Supprimer un compte 'point de vente'.
        * Ajouter un prospect.
    - Gérer ses objectifs de vente :
        * Créer un objectif .
        * Modifier un objectif.
        * Supprimer un objectif.
    - Ajouter les concurrents :
        * Ajouter des informations sur un concurrent dans la base de donnée.
        * Ajouter un concurrent sur la carte.
    - Consulter l'état de stock.
    - Consulter les promos et les actualités de l'entreprise.
    - Consulter les réclamations envoyer par les agents point de vente.

2.2.2 Application web

Cette application sera destinée pour l'admin. Elle a pour objectif de gérer les actions réalisé par les utilisateurs de l'application mobile.
L'Admin hérite tous les cas d'utilisation de l'agent commercial.

- **Un Admin peut** :
- S'authentifier.
- Gérer les actualités :
    - Créer une actualité .
    - Modifier une actualité.
    - Supprimer une actualité.
    - Consulter les actualités.
- Traiter les réclamations.

- Consulter les états de stock.
- Gérer les comptes.
  - Créer un compte.
  - Modifier un compte.
  - Supprimer un compte.
  - Consulter les comptes.
- Gérer les rôles.

2.3 Besoins non fonctionnels

Un besoin non fonctionnel est une exigence pour assurer un ou plusieurs services de notre application, et qui doit être respecté pour garantir une solution appropriée aux attentes des concepteurs.

Notre application doit nécessairement confirmer ces besoins :

- **Ergonomie et expérience utilisateur** :
  - **Interfaces conviviales :** Les applications doivent posséder des interfaces avec des couleurs bien choisit et une présentation claire.
  - **Formulaires de gestion simples :** Tant qu'il s'agit d'une solution destinée aux grands producteurs de produits et puisque les activités de gestion sont fréquentes et répétitives,il faut que le système offre des formulaires simples.
  - **Mise en page :** Très fréquemment, l'utilisateur effectuera une série de workflows avec un ordre spécifique. Pour cette raison, l'affichage des sections doit respecter cet ordre dans le but d'avoir une gestion plus accélérée.
- **Sécurité** : Parmi les facteurs les plus importants de notre solution est la sécurité, ce qui est assuré par l'intégration du protocole oauth2 en utilisant la solution d'authentification et autorisation Keycloack.

- **L'extensibilité** : Notre application comporte plusieurs modules et fonctionnalités ce pour cela nous avons investi plein de temps pour choisir une architecture bien étudiée afin de bien organiser le travail et répondre à certains changements futurs.

- **La performance** : Nous savons tous qu'une application qui offre des performances médiocres peut ralentir la productivité, alors le temps de réponse des différentes fonctionnalités de notre application doit être presque idéal, c'est pour cela que nous devons optimiser le maximum le code et les requêtes de notre application lors du développement ainsi que les réponses demandés au serveur. Dans notre projet Les entreprises qui utiliseront notre système peuvent avoir des centaines et des milliers d'utilisateur entre vendeurs, responsables et commerciaux. Le système doit donc supporter :

  - La gestion de plus de 10000 comptes.
  - La connexion simultanée de plus de 1000 utilisateurs.
  - Les opérations de création simples telles que l'ajout, la suppression, la modification ne doivent pas prendre plus de 1 seconde.

- **Fiabilité** : Pour inspirer la confiance et le professionnalisme à nos futurs clients avant un achat ou une transaction sur internet, notre solution doit être fiable au niveaux de la conformité au règlement, la sécurité des données et la haute disponibilité, etc.

2.4 Diagrammes de cas d'utilisation

Nous avons cité si dessus les acteurs principaux de notre solution, par la suite nous présentons les différentes fonctionnalités offerte à eux. Pour cela, nous avons annoncé quelques diagrammes de cas d'utilisation comme objectif de faciliter la compréhension du déroulement des actions par les acteurs.
Dans cette rubrique on va mieux s'intéresser à éclaircir les besoins fonctionnel des acteurs agent de point de vente et agent commercial.

2.4.1 Diagramme de cas d'utilisation générale

Les étapes essentielles de notre projet sont détaillées dans la figure 2.1, le diagramme du cas d'utilisation générale nous facilite la conception des autres cas d'utilisation et il nous éclaircit les différentes interactions de chaque acteur.
La figure 2.1 présente le diagramme de cas d'utilisation générale.

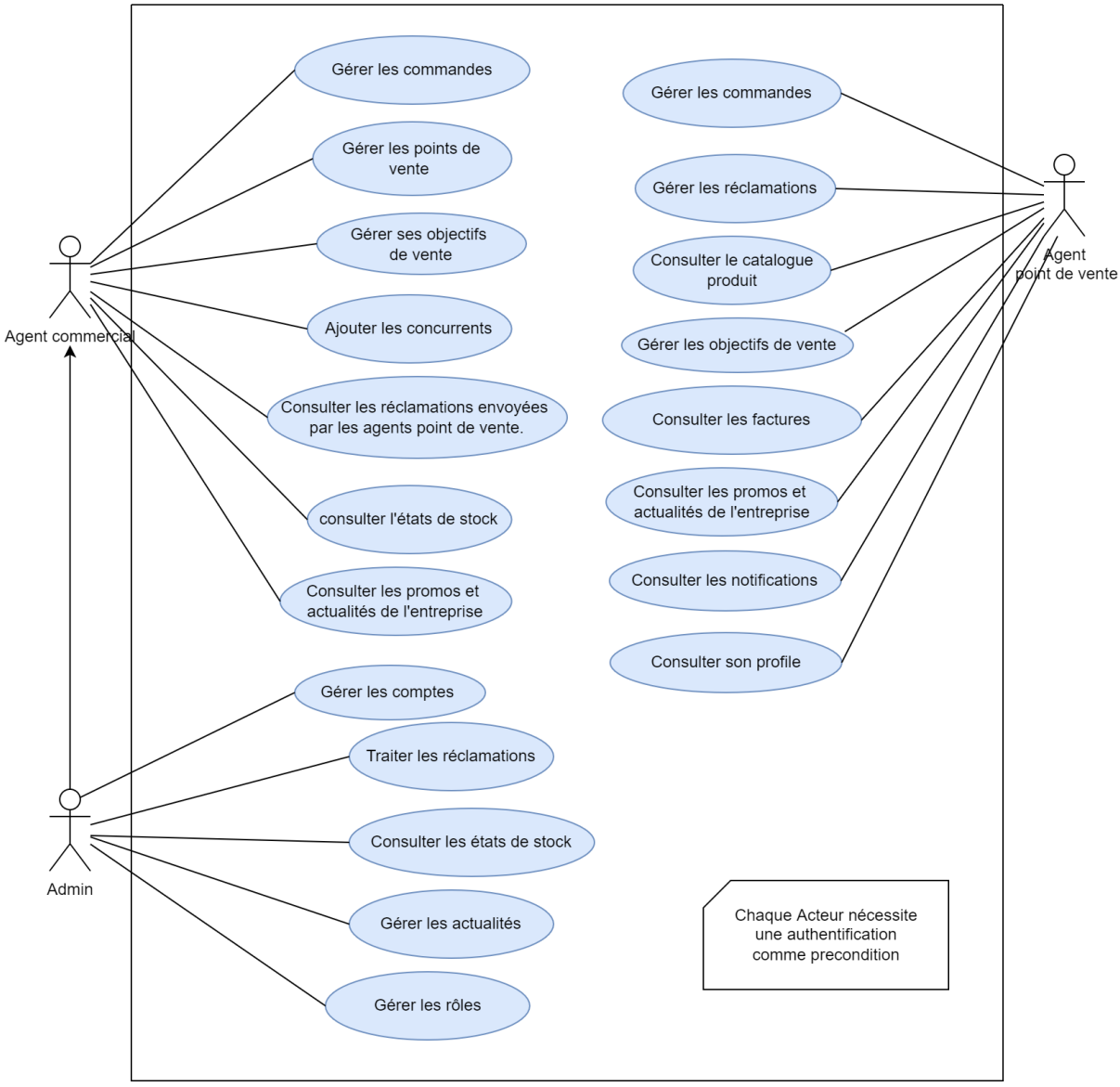


FIGURE 2.1 – Diagramme du cas d'utilisation générale

2.4.2 Diagramme de cas d'utilisation de l'agent de point de vente

La figure 2.2 présente le diagramme de cas d'utilisation de l'agent de point de vente.



FIGURE 2.2 – Diagramme du cas d'utilisation de l'agent de point de vente

2.4.3 Diagramme de cas d'utilisation de l'agent commercial

La figure 2.3 présente le diagramme de cas d'utilisation de l'agent commercial.

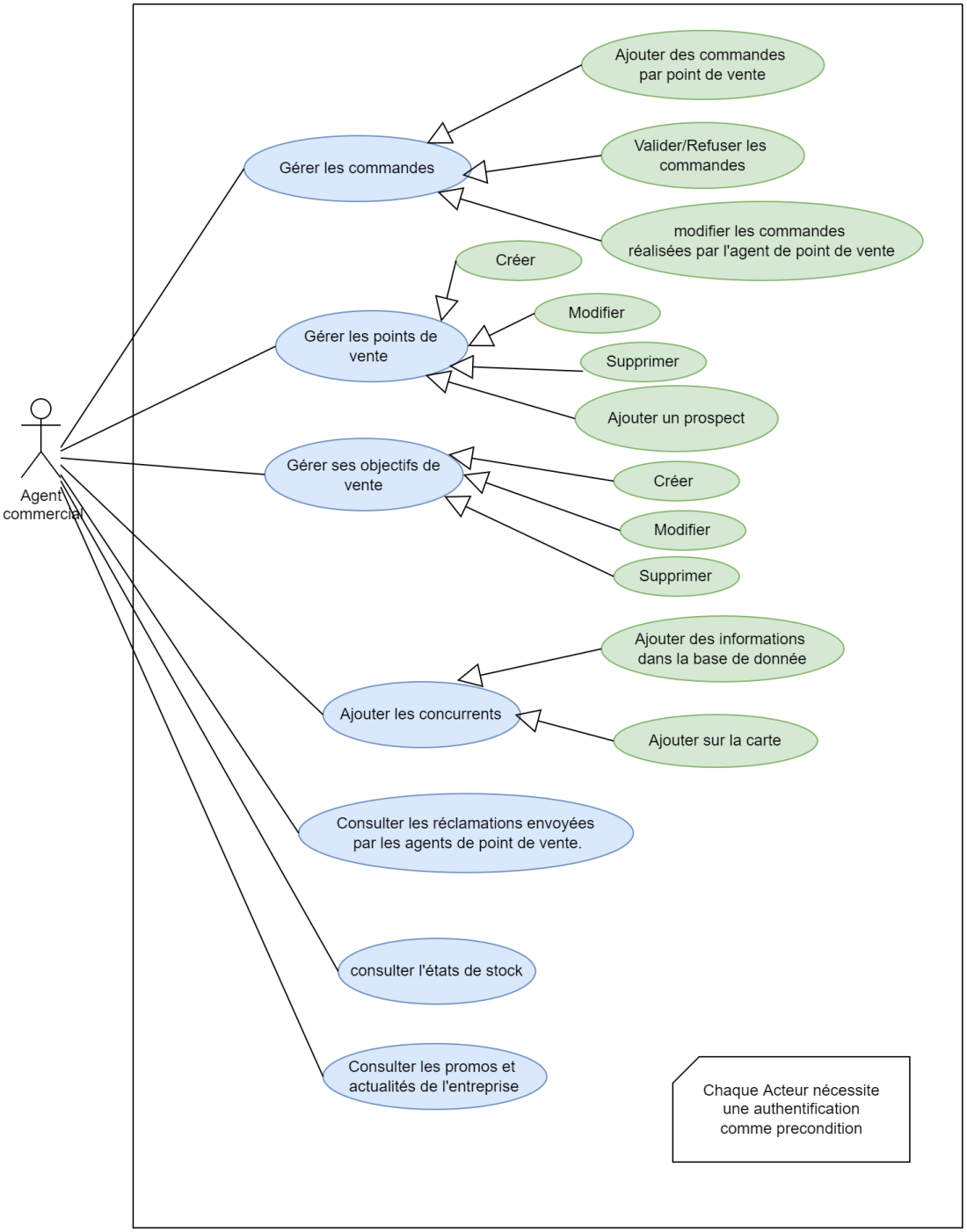


FIGURE 2.3 – Diagramme du cas d'utilisation de l'agent commercial

2.4.4 Diagramme de cas d'utilisation de l'Admin

La figure 2.4 présente le diagramme de cas d'utilisation de l'admin de la solution.

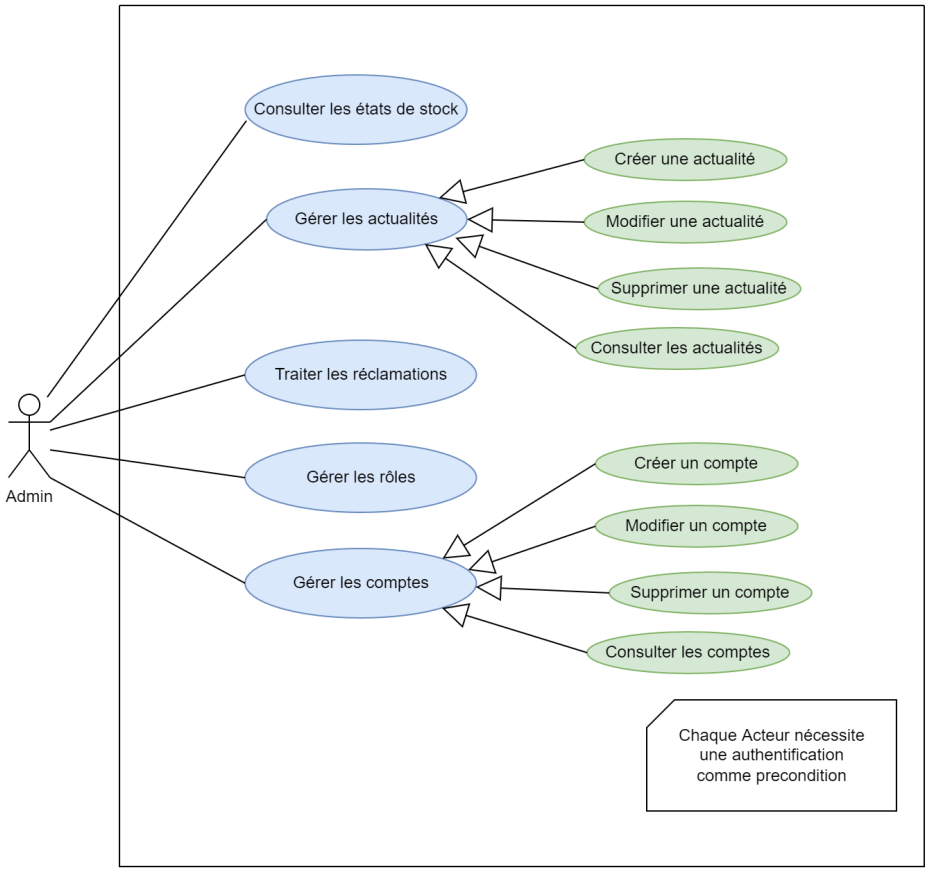


FIGURE 2.4 – Diagramme du cas d'utilisation de l'admin de la solution

2.5 Planification des tâches

ID	User Story	Estimation (j)	Priorité
1	En tant qu'utilisateur je veux m'authentifier Pour accéder à mon espace.	8	Haut
2	En tant qu'utilisateur je veux me déconnecter Pour fermer ma session.	4	Haut
3	En tant qu'agent PV je veux consulter mon profil	5	Haut
4	En tant qu'agent PV je veux consulter mes notifications.	5	Haut
5	En tant qu'agent PV/ agent commercial/admin je veux consulter les promos et actualités de l'entreprise. Pour rectifier des détails	9	Haut
6	En tant qu'agent PV je veux Consulter les factures.	5	Haut
7	En tant qu'agent PV je veux gérer les objectifs de vente.	6	Haut
8	En tant qu'agent PV je veux consulter le catalogue produit	7	Haut
9	En tant qu'agent PV je veux gérer les réclamations	6	Haut
10	En tant qu'agent PV je veux gérer les commandes	7	Haut
11	En tant que commercial je veux consulter l'états de stock	10	Moyenne
12	En tant que commercial/admin je veux consulter les réclamations envoyer par les agents point de vente.	4	Moyenne
13	En tant que commercial/admin je veux ajouter des concurrents	6	Faible

14	En tant que commercial/admin je veux gérer mes objectif de vente	5	Moyenne
15	En tant que commercial/admin je veux gérer les points de vente	6	Moyenne
16	En tant que commercial/admin je veux Gérer les commandes	7	Moyenne
17	En tant qu'admin je veux gérer les comptes	10	Haut
18	En tant qu'admin Je veux traiter les réclamations	3	Moyenne
19	En tant que commercial je veux consulter les états de stock	5	Faible
20	En tant qu'admin je veux gérer les actualités	6	Faible
21	En tant qu'admin Je veux gérer les rôles	4	Haut

TABLE 2.2 – Product backlog

Conclusion

Au court de ce chapitre nous avons fait l'étude des différents besoins fonctionnels et non fonctionnels des utilisateurs. Nous avons aussi conçu les diagrammes des cas d'utilisations pour chaque acteur. A cette phase, nous avons pu avoir une idée générale sur le fonctionnement global de l'application de point de vue des utilisateurs et détaillé les fonctionnalités principales de notre application, ensuite nous allons attaquer la conception. Dans le chapitre suivant.

Chapitre 3

Conception

3.1 Introduction

Après avoir décrit les différentes fonctionnalités de notre solution, nous nous concentrerons dans ce chapitre à la phase de conception. Nous définirons dans un premier temps l'architecture générale de note système, ensuite, nous présenterons les patrons de conception que nous avons exploités. Puis, nous dresserons le diagramme de classes et les diagrammes de séquences pour chaque package.

3.2 Architecture 3-tiers

Lors de la conception d'un système logiciel, il est très important de choisir judicieusement le modèle d'architecture qui lui convient, ceci garantira le bon fonctionnement entre les différentes couches.
Pour la réalisation de notre solution, nous avons opté pour l'architecture 3-tiers qui est un modèle logique visant à séparer nettement trois couches logicielles au sein d'une même solution. Les trois niveaux de cette architecture sont décrits brièvement comme suit :

- **Couche présentation :** Il s'agit de la partie visible de l'application qui permet à l'utilisateur d'interagir avec le système.
- **Couche applicative :** C'est la partie qui implémente la logique, les différentes règles de gestion et qui décrit les opérations que l'application effectue sur les

données en fonction des requêtes des utilisateurs réalisées au travers de la couche présentation.

- **Couche accès aux données :** C'est la couche qui gère l'accès aux données. Ces données sont récupérées et retournées à la couche applicative pour être traitées et renvoyées à l'utilisateur.

La figure 3.1 illustre les interactions entre les différents composants de l'architecture 3-tiers .

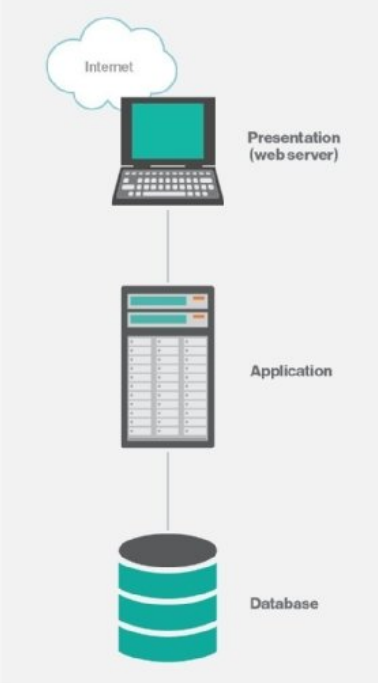


FIGURE 3.1 – Architecture 3-tiers [1]

3.3 Patrons de conception

Au cours du développement des applications, nous nous rencontrons souvent à des problèmes de conception récurrents. Pour les résoudre de manière efficace et garantir la réutilisabilité nous utilisons les patrons de conception. Ces patrons sont des descriptions ou des modèles reconnus comme étant une convention à suivre permettant non seulement de corriger le problème mais aussi d'améliorer la qualité du code et de bien organiser le processus de développement. Dans notre cas nous avons utilisé deux patrons de conception selon la couche à développer.

3.3.1 Patron de conception clean architecture

Tant que notre application mobile contient beaucoup des fonctionnalités diversifiées nous avons choisit de travaillier avec le fameux patron de conception clean d'architecture qui'est une catégorie de modèle de conception logicielle qui suit des concepts de clean code et met en œuvre des principes SOLID.
Il s'agit essentiellement d'un ensemble de principes de conception des meilleures pratiques qui nous aident à conserver la logique métier, ou la logique de domaine ensemble et à minimiser les dépendances au sein du système. Si elles sont suivie, une architecture propre permet aux architectes logiciels de découpler les composants afin qu'ils deviennent suffisamment isolés pour être durables et facilement modifiables sans refaire le système.
Le concept de code propre et d'architecture propre a été initialement inventé en 2008 par Robert C. Martin, connu dans la communauté sous le nom d'"Oncle Bob". Son livre 'The Clean Coder [4]' est devenu un guide de base pour la conception d'applications évolutives modernes dont les quatre principaux composants sont les suivants :

- **Entities (Entités) :** Une entité est modèle métier de l'application ou du système à construire. Une entité peut être un objet, une structure de données avec des fonctions.

- **Use cases (Cas d'utilisation) :** Un cas d'utilisation est une fonctionnalité métier. C'est dans cette couche qu'on implémente les fonctionnalités métiers de l'application.

- **Interface Adapters (Adaptateurs d'interface) :** Cette couche fait la liaison entre les couches supérieures et les couches inférieures. Les adaptateurs convertissent les données dans les formats adaptés aux cas d'utilisation et aux entités. Inversement, les adaptateurs convertissent également les données reçues des cas d'utilisations dans les formats adaptés aux frameworks et autres dépendances externes. Les opérations de base de données, comme les requêtes SQL par exemple sont implémentées dans cette couche.

- **Framework and Drivers (Frameworks et Pilotes) :** Dans les clean architecture, les outils comme les frameworks, bases de données sont considérés

comme des détails. Tout ce qui est lié aux frameworks et dépendances externes est également défini dans cette couche.

La figure 3.2 présente le principe du patron de conception clean architecture selon Uncle Bob.

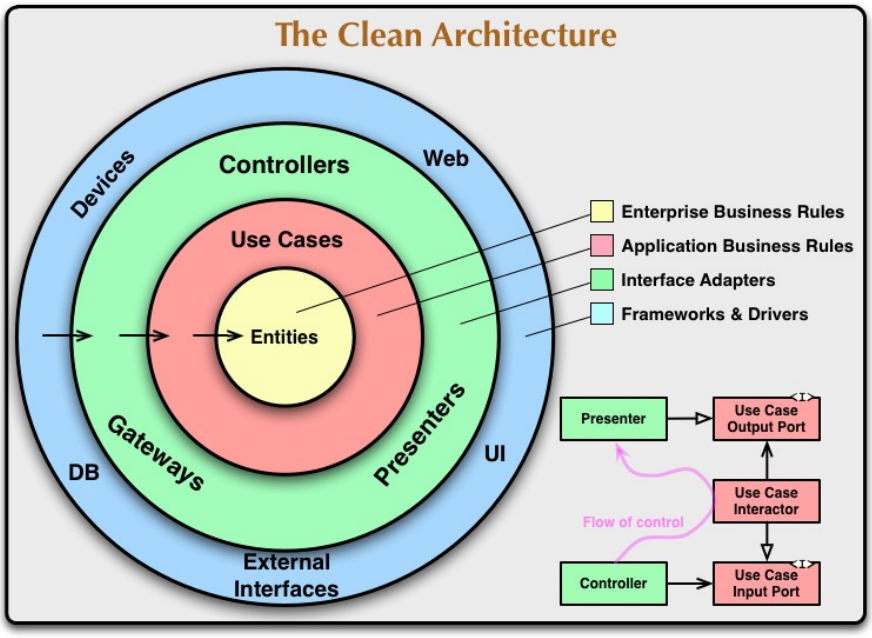


FIGURE 3.2 – Clean architecture d'après Uncle Bob [3]

Dans la figure 3.3 nous présentons un extrait de l'architecture de l'application mobile "Inetum Force de vente".

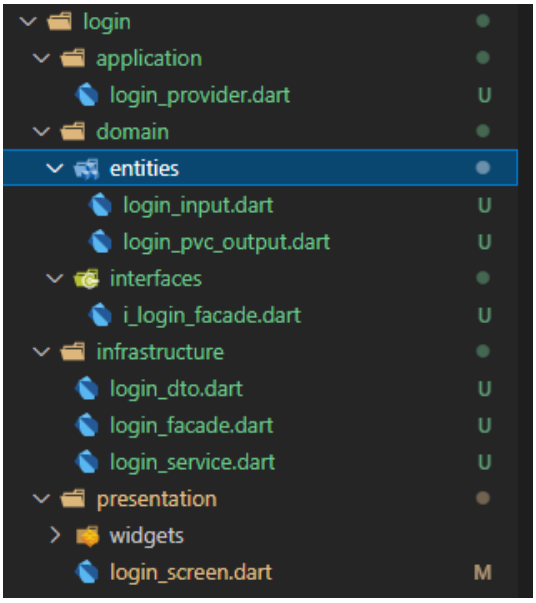


FIGURE 3.3 – Extrait de l'architecture de l'application mobile "Inetum Force de vente"

3.3.2 Patron de conception MVC

Dans la partie backend et web nous avons choisit le Model-View-Controller (MVC) qu'est l'un des patrons de conceptions les plus populaires et les plus utilisés lors du développement des applications. Il a commencé à être utilisé depuis l'année 1978 pour développer des clients lourds mais grâce à sa puissance il a été adopté rapidement pour le développement des applications web. Ce patron de conception répond aux besoins des applications interactives en séparant les problématiques liées aux différents composants au sein de leur architecture respective. Les trois principaux composants sont les suivants :

- **Modèle :** Il contient les données pures de l'application, il ne contient aucune logique décrivant comment présenter les données à l'utilisateur.
- **Vue :** Elle permet de présenter et de mettre en forme les données du modèle pour l'utilisateur
- **Contrôleur :** Il se place entre la vue et le modèle. Il détecte les événements déclenchés et exécute la réaction appropriée à ces derniers.

La figure 3.4 présente le principe du patron de conception MVC.

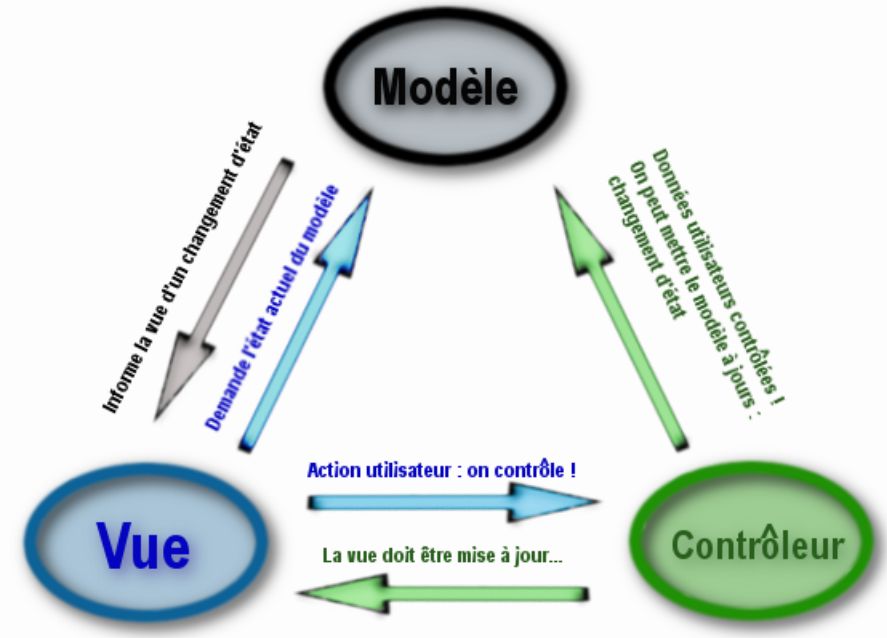


FIGURE 3.4 – Principe du patron de conception MVC [19]

Dans la figure 3.3 nous présentons un extrait de l'architecture de l'architecture du backend de l'application "Inetum Force de vente".

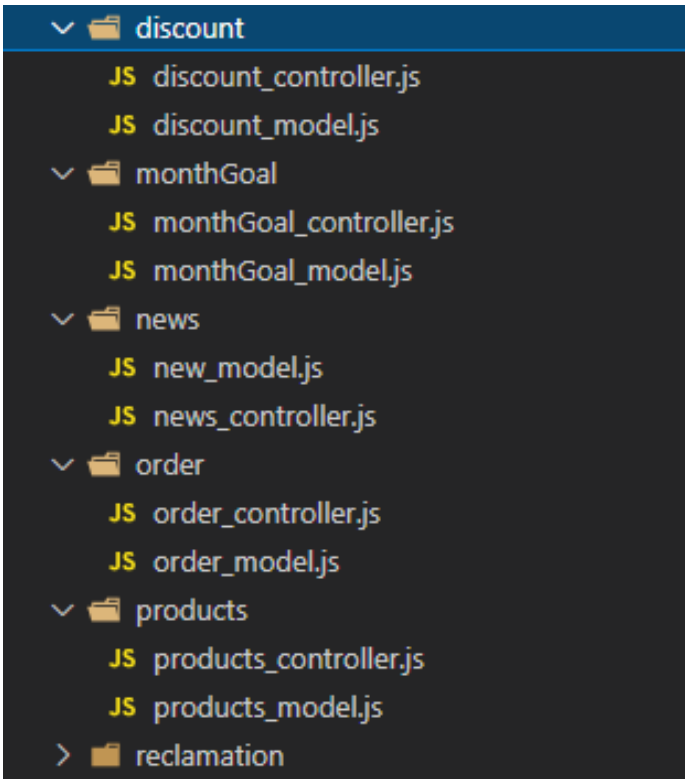


FIGURE 3.5 – Extrait de l'architecture du backend de l'application "Inetum Force de vente"

3.4 Conception détaillée

Afin d'obtenir une vue claire et complète sur la structure interne du système, nous allons, en premier lieu présenter le diagramme de classe des entités de la solution entière, puis faire détailler la conception de cas d'utilisation le plus important à l'aide des diagrammes de séquences.

Ce diagramme est toujours considéré comme le diagramme le plus important dans la modélisation orientée-objet. Il s'agit d'une représentation statique des éléments composant un système avec leurs attributs et leurs opérations ainsi que les différentes relations entre eux.

Le tableau 3.1 donne une description brève de différents classes.

Num	Classe	Description
1	Catégorie	Modélise les informations relatives à une catégorie de produits.
2	Commande	Décrit les données relatives à une commande.
3	Actualité	Présente les informations relatives aux actualités de l'entreprise.
4	Facture	Contient les informations qui concernent une facture tels que taxes, montant, etc.
5	Image	Contient les images des produits.
6	LigneCommande	Possède les informations concernant une ligne de commande avec le produit et sa quantité.
7	LigneFacture	Présente les données d'une ligne de facture avec le produit, la quantité et le prix de vente unitaire.
8	PointDeVente	Présente les détails d'un point de vente.
9	Produit	Définit les informations qui concernent un produit.
10	Utilisateur	Modélise les détails générale d'un utilisateur de l'application Inetum Force de vente.
11	AgentCommercial	Présente les informations relatives au commercial.
12	AgentPointDeVente	Modélise les détails d'un responsable d'un point de vente.
13	Taxe	Définit le taux de taxe à appliquer à une catégorie.
14	Distribution	Contient les détails d'une distribution d'une commande.
15	Véhicule	Décrit le véhicule qui sera utilisé pour une distribution.
16	Entrepôt	Modélise les informations relatives à un entrepôt.
17	Réclamation	Contient les informations relatives à une réclamation.
18	ObjectifMensuelle	Modélise les objectifs de vente mensuelles souhaité par un commercial ou un agent point de vente .
19	ProduitEnPromo	définit le nouveau prix d'un produit sous promos.

TABLE 3.1 – Tableau descriptif du diagramme de classes des entités

Le diagramme de classes de notre système est illustré par la figure 3.6.

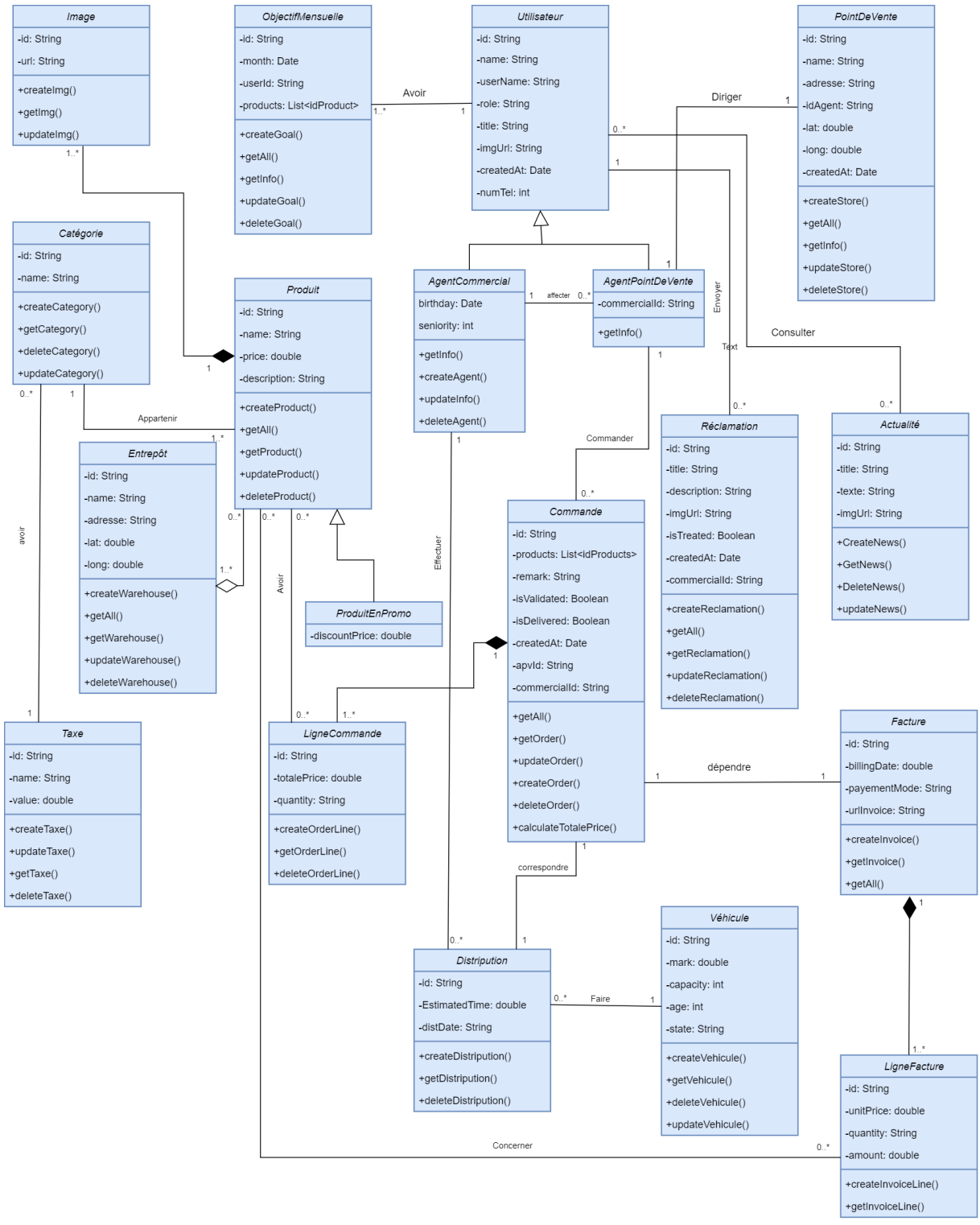


FIGURE 3.6 – Diagramme de classes des entités

3.4.1 Diagrammes de séquences

Les diagrammes de séquence (DS) servent à décrire en détail la façon dont les éléments du système interagissent entre eux en précisant les messages échangés suivant un ordre chronologique . Nous allons donc exploiter ce type de diagramme dans le but de décrire des cas d'utilisations.

DS du cas d'utilisation «Authentification»

Dès où l'un des points forts de notre solution, c'est l'intégration d'une solution SSO (Single Sign On) qui permette à nos utilisateurs d'authentifier à toutes nos plate-forme et solution en utilisant les mêmes identifiants. Dans notre projet si l'utilisateur souhaite s'authentifier il accède a l'interface de login et remplit le formulaire, si le contrôle de champs et vérifié une requête HTTP POST envoie le donnée au contrôle de "l'identity provider" keycloak que vérifie le droit d'accès de l'utilisateur et renvoie un HTTP SUCCESS avec un accessToken et des information supplémentaire sur l'acteur.
Nous allons représenter dans la figure 3.7 le scénario nominal d'authentification.

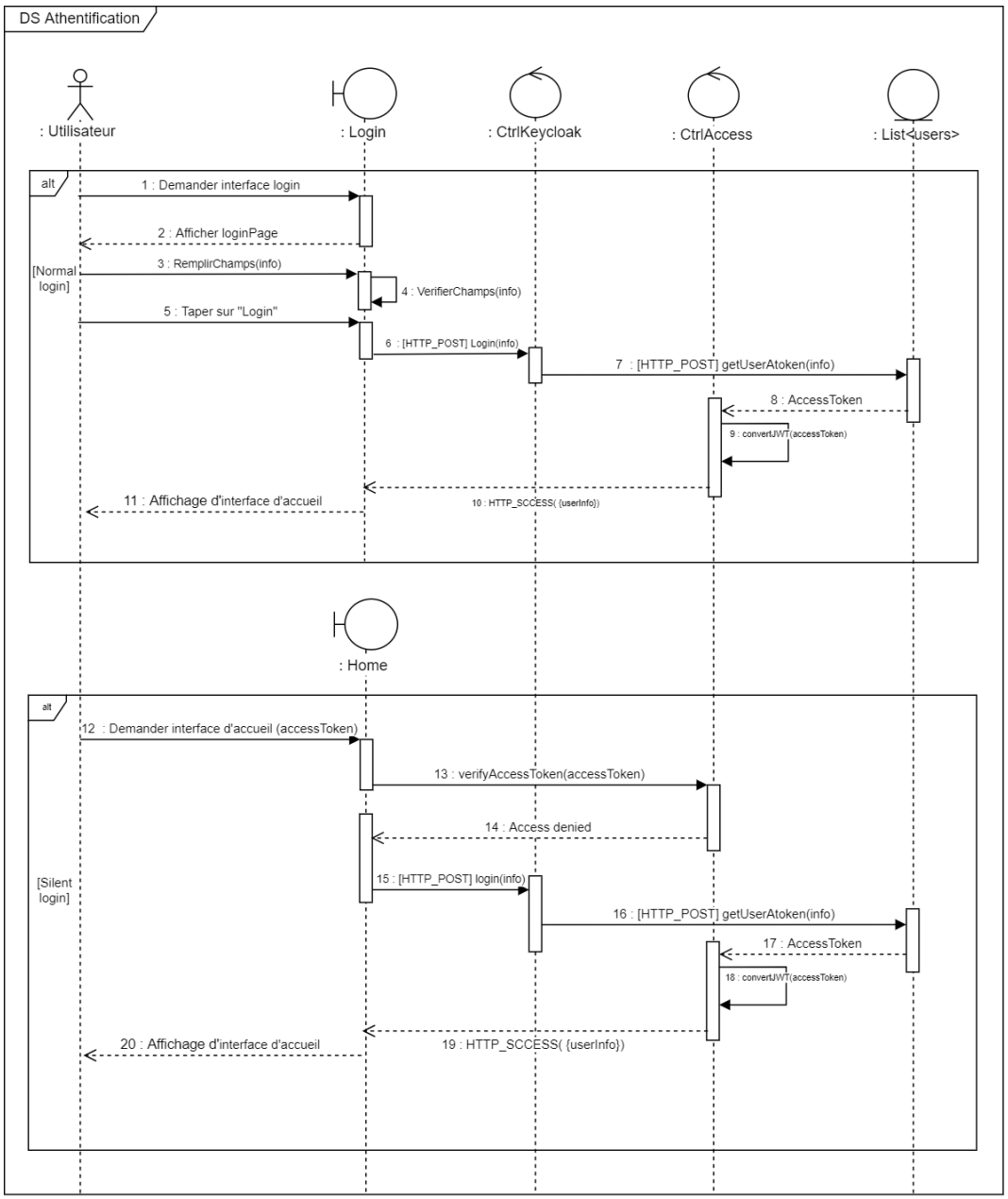


FIGURE 3.7 – DS du cas d'utilisation «Authentification»

DS du cas d'utilisation «Gérer réclamation»

Parmi les principes le plus important du principe de force de vente c'est l'écoute et l'interaction rapide avec les problème des clients, ce pour cela si le client décide d'envoyer une réclamation, il accède au formulaire puis il doit le remplir avec les informations nécessaire pour le traitement en respectant les conditions de saisie, Si les valeurs entrées sont correctes, une requête est envoyée au contrôle. Après une création réussie, une réponse HTTP CREATED est envoyé et un message de succès est affiché sur l'écran.

Si l'acteur souhaite consulter la liste des catégories, le système les récupère depuis l'entité et renvoie une réponse HTTP OK contenant cette liste qui sera affichée pour l'utilisateur.

Après avoir traiter la réclamation dans la vie réel le commercial peut changer son état de traitement et notifier l'agent de point de vente du changement .

Nous allons représenter dans la figure 3.8 le scénario nominal de gestion de réclamation.

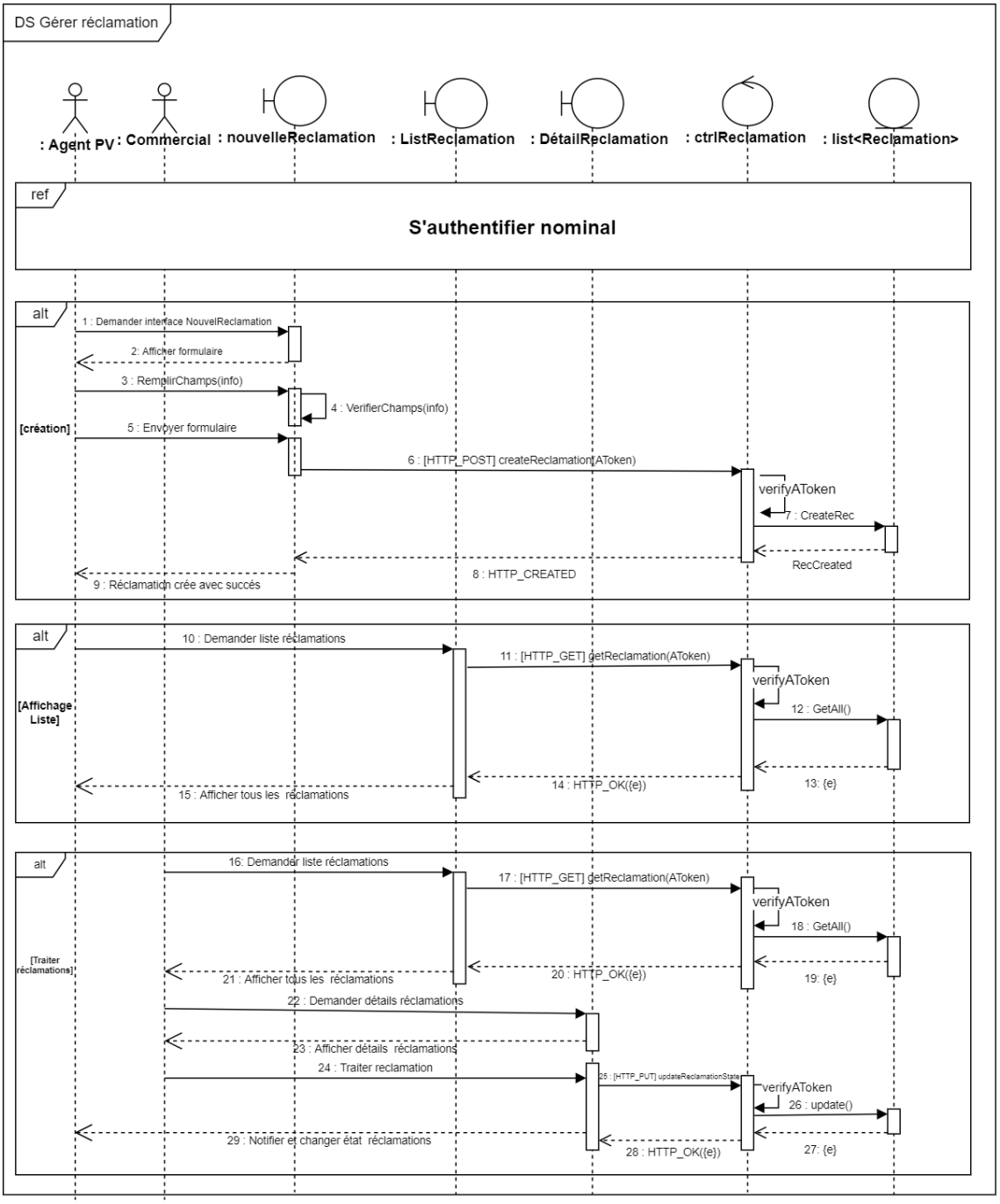


FIGURE 3.8 – DS du cas d'utilisation «Gérer réclamation»

DS du cas d'utilisation «Gérer commandes»

À fin d'assurer la satisfaction de clients et le bon fonctionnement de l'application il faut que l'une de principales fonctionnalités qu'est la gestion de commande soit facile, sécurisé et temps réel, donc lors de la création de commande l'agent de point de vente accède à l'interface de sélection des produits. Lorsqu'il choisit un produit, un formulaire s'affiche pour indiquer la quantité et une remarque, en tapant sur valider la commande sera sauvegarder dans le panier, une fois l'agent finit ses achats, il accède à l'interface de panier et envoie la commande, une requête est envoyée au «ctrlcommande» qui vérifie l'access token de l'agent et sauvegarde la commande a la base de données.
Le commercial accède à l'interface de validation de commande puis choisit les commandes à valider en modifiant son état de "en attente" à "valider".
Nous allons représenter dans la figure 3.9 le scénario nominal de gestion de commandes.

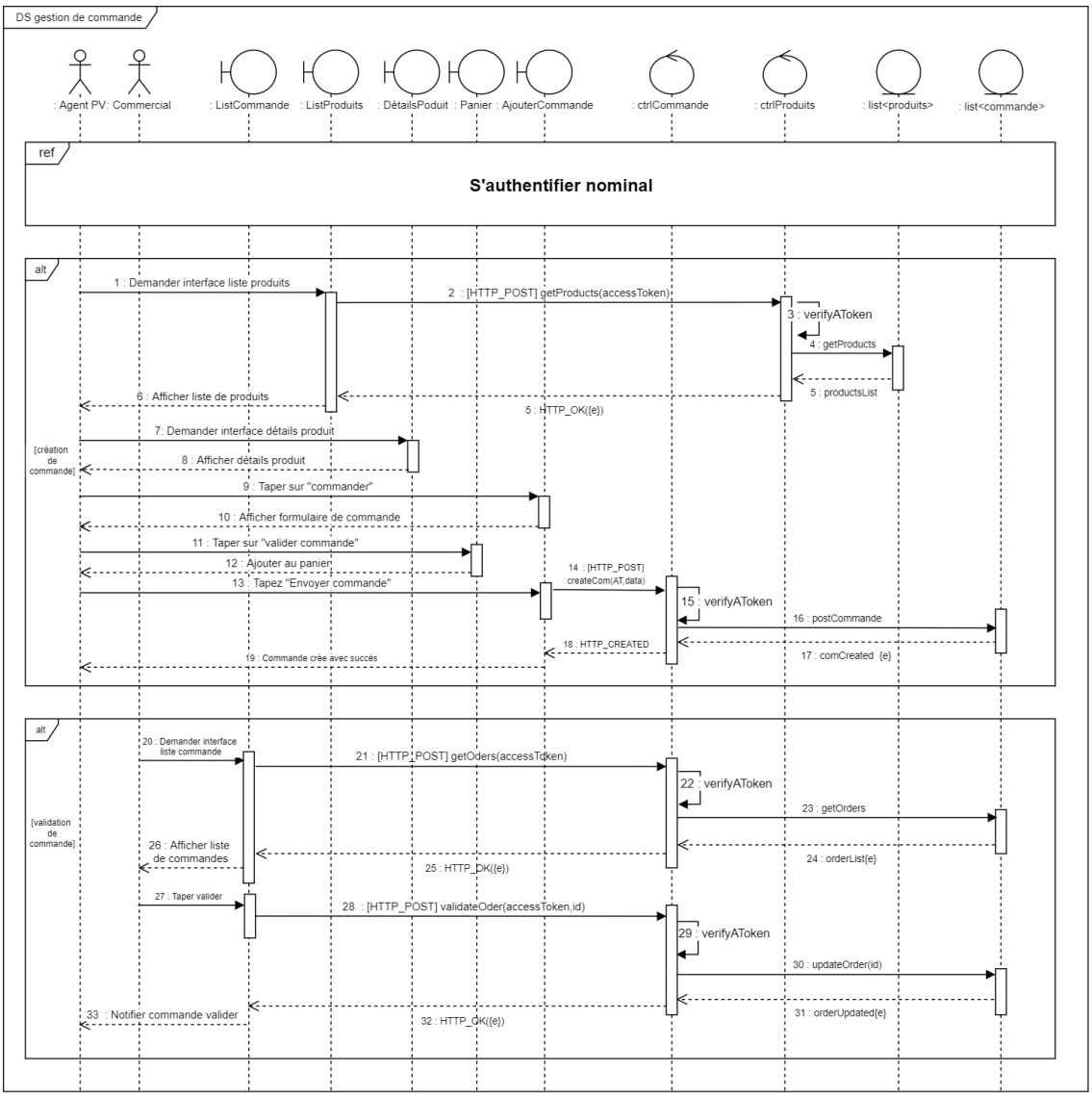


FIGURE 3.9 – DS du cas d'utilisation «Gérer commandes»

3.4.2 Architecture logicielle générale

Parmi les principales fonctionnalités de notre application Inetum force de vente ce qu'elle communique directement avec l'ERP(sageX3 ...) des entreprises que assure une synchronisation et un échange des données temps réelle.

Notre application est divisé sur trois couche principale.

la figure 3.10 représente l'architecture générale de l'application Inetum Force de vente.

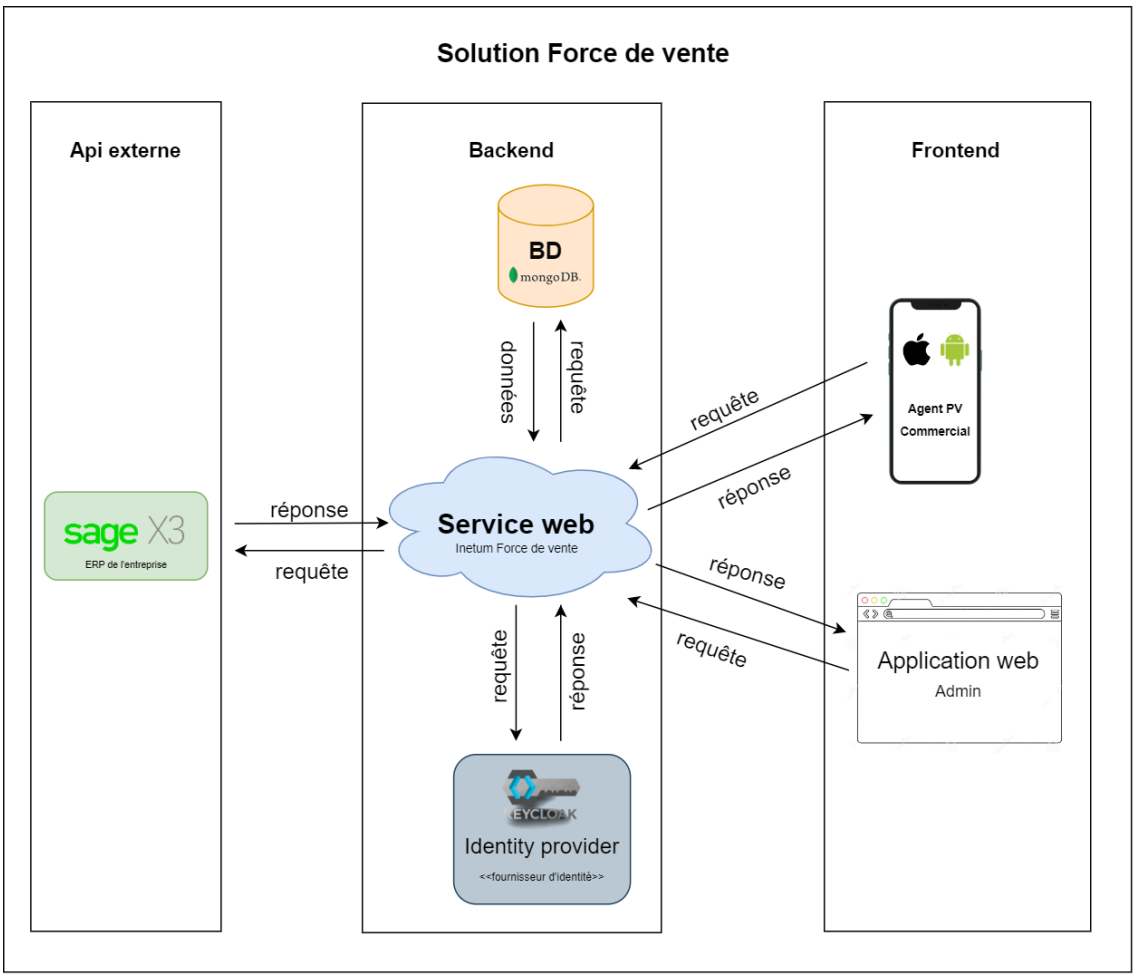


FIGURE 3.10 – Architecture logicielle générale

Conclusion

Au cours de ce chapitre, nous avons défini l'aspect statique et l'aspect dynamique de notre système grâce au diagramme de classe et les diagrammes de séquences. Dans le prochain chapitre, nous allons présenter l'architecture physique, les outils d'intégrations et quelques interfaces de notre application.

Chapitre 4

Réalisation

4.1 Introduction

Au cours de ce dernier chapitre, nous mettrons l'accent sur la phase finale du projet qu'est la réalisation.
Ce dernier chapitre est le fruit du travail réalisé aux chapitres précédents.
Cependant, après avoir discuté de nos choix conceptuels, nous allons dans ce chapitre déterminer les technologies utilisées, les outils du développement utilisés et les langages de programmation, par le diagramme de déploiement de la solution.
Pour finir, nous allons présenter notre application via des captures d'écran des différents interfaces Homme/Machine et nous clôturons par une conclusion résumant les principales idées décrite précédemment.

4.2 Environnement de développement

Dans cette section nous décrivons les frameworks et les logiciels exploités pour réaliser notre solution.

4.2.1 Technologies utilisées

Au cours de cette sous-section nous décrivons les différents frameworks et environnement exploités pour concevoir notre solution.

Flutter

Flutter [12] est un kit de développement logiciel (SDK) d'interface utilisateur open-source créé par Google. Il est utilisé pour développer des applications pour Android, iOS, Linux, Mac, Windows, Google Fuchsia et le web à partir d'une seule base de code.



Node.js

Node.js [20] est une plateforme logicielle libre en JavaScript, orientée vers les applications réseau évènementielles hautement concurrentes qui doivent pouvoir monter en charge. Elle utilise la machine virtuelle V8, la librairie libuv pour sa boucle d'évènements, et implémente sous licence MIT les spécifications CommonJS.
Concrètement, Node.js est un environnement bas niveau permettant l'exécution de JavaScript côté serveur.



Express.js

Express.js [10] est un framework pour construire des applications web basées sur Node.js4. C'est de fait le framework standard pour le développement de serveur en Node.js5. L'auteur original, TJ Holowaychuck, le décrit comme un serveur inspiré de Sinatra6 dans le sens qu'il est relativement minimaliste tout en permettant d'étendre ses fonctionnalités via des plugins.



Provider

Le package Provider [23] , créé par Remi Rousselet, vise à gérer l'état le plus proprement possible. Dans le fournisseur, les widgets écoutent les changements d'état et se mettent à jour dès qu'ils sont notifiés.

MongoDB

MongoDB [17] (de l'anglais humongous qui peut être traduit par « énorme ») est un système de gestion de base de données orienté documents, répartissable sur un nombre quelconque d'ordinateurs et ne nécessitant pas de schéma prédéfini des données. Il est écrit en C++. Le serveur et les outils sont distribués sous licence SSPL, les pilotes sous licence Apache et la documentation sous licence Creative Commons. Il fait partie de la mouvance NoSQL.



4.2.2 Langages de programmation

Dart

Dart [8] est un langage de programmation optimisé pour les applications sur plusieurs plateformes. Il est développé par Google et est utilisé pour créer des applications mobiles, de bureau, de serveur et web. Dart est un langage orienté objet à ramasse-miettes avec une syntaxe de type C++. Dart peut se compiler en code natif ou en JavaScript. Il prend en charge les interfaces, les mixins,les classes abstraites, les génériques réifiés et l'inférence de type.



JavaScript

JavaScript [15] est un langage de programmation de scripts principalement employé dans les pages web interactives et à ce titre est une partie essentielle des applications web. Avec les langages HTML et CSS, JavaScript est au cœur des langages utilisés par les développeurs web. Une grande majorité des sites web l'utilisent, et la majorité des navigateurs web disposent d'un moteur JavaScript pour l'interpréter.
JavaScript est aussi employé pour les serveurs Web avec l'utilisation (par exemple) de Node.js ou de Deno.



4.2.3 Outils

Figma

Figma [11] est un éditeur de graphiques vectoriels et un outil de prototypage. Il est principalement basé sur le web, avec des fonctionnalités hors ligne supplémentaires activées par des applications de bureau pour macOS et Windows.



Postman

Postman [22] est une application permettant de tester des API, créée en 2012 par Abhinav Asthana, Ankit Sobti et Abhijit Kane à Bangalore pour répondre à une problématique de test d'API partageable. D'abord module complémentaire de Google Chrome, puis client lourd, et finalement client léger, elle est à présent utilisée par plus de 500 000 entreprises dans le monde et a son siège à San Francisco



Heroku

Heroku [14] est une entreprise créant des logiciels pour serveur qui permettent le déploiement d'applications web. Créée en 2007, elle est rachetée en 2010 par l'éditeur Salesforce.com.



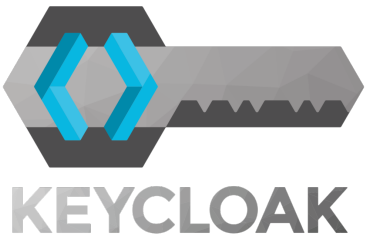
MongoDB Atlas

MongoDB Atlas [18] est un service de base de données multi-cloud par les mêmes personnes qui ont construit MongoDB. Atlas simplifie le déploiement et la gestion de vos bases de données tout en offrant la polyvalence dont vous avez besoin pour créer des applications globales résilientes et performantes sur les fournisseurs de cloud de votre choix.



Keycloak

Keycloak [16] est un logiciel à code source ouvert permettant d'instaurer une méthode d'authentification unique à travers la gestion par identité et par accès. Initialement développé par les équipes JBoss, le projet est depuis mars 2018 sous la gérance de Red Hat qui l'utilise en amont de sa solution RH-SSO2. Keycloak se définit comme une application rendant possible la sécurisation de n'importe quelle application web moderne avec un apport minimum en termes de code.



Cloud IAM

Cloud IAM [6] automatise chaque partie de la configuration, de l'exécution et de la mise à l'échelle des clusters Keycloak. Disponible sur toutes les principales plates-formes cloud et applicatives du monde entier.



Cloudinary

Cloudinary [7] est une société de technologie SaaS basée à Santa Clara, en Californie, avec des bureaux en Angleterre, en Pologne et à Singapour. La société fournit des services de gestion d'images et de vidéos basés sur le cloud. Il permet aux utilisateurs de télécharger, stocker, gérer, manipuler et diffuser des images et

des vidéos pour des sites Web et des applications.



Visual Studio Code

Visual Studio Code [25] est un éditeur de code extensible développé par Microsoft pour Windows, Linux et macOS.



Android Studio

Android Studio [2] est un environnement de développement pour développer des applications mobiles Android. Il est basé sur IntelliJ IDEA et utilise le moteur de production Gradle. Il peut être téléchargé sous les systèmes d'exploitation Windows, macOS, Chrome OS et Linux.



diagrams.net

diagrams.net [9] (anciennement draw.io) est un logiciel de dessin graphique multiplateforme gratuit et open source développé en HTML5 et JavaScript. Son interface peut être utilisée pour créer des diagrammes tels que des organigrammes, des structures filaires, des diagrammes UML, des organigrammes et des diagrammes de réseau.



GitHub

GitHub [13] est un service web d'hébergement et de gestion de développement de logiciels, utilisant le logiciel de gestion de versions Git.



ClickUp

ClickUp [5] est une plateforme de productivité qui permet aux organisations de gérer des projets, du temps et du travail.



Overleaf

Overleaf [21] est un éditeur LaTeX collaboratif basé sur le cloud utilisé pour écrire, éditer et publier des documents scientifiques. Il s'associe à un large éventail d'éditeurs scientifiques pour fournir des modèles LaTeX de journaux officiels et des liens de soumission directs.



4.3 Diagramme de déploiement

Le diagramme de déploiement permet de modéliser le déploiement physique des données créées par le système sur des composants matériels en illustrant la manière dont ces derniers sont répartis et les relations entre eux. Avec ce type de diagramme nous arrivons à décrire la topologie de notre système matériel. Dans notre cas, pour créer ou récupérer des données, l' application mobile doivent se connecter à l'API web reliée à un serveur de base de données, l'ERP de l'entreprise et un fournisseur d'identité(Identity provider). La transmission et la récupération des données se fait par l'intermédiaire du composant « Service » en relation avec le composant « Route » de l'API développée avec Express.js comme présenté par la figure suivante.

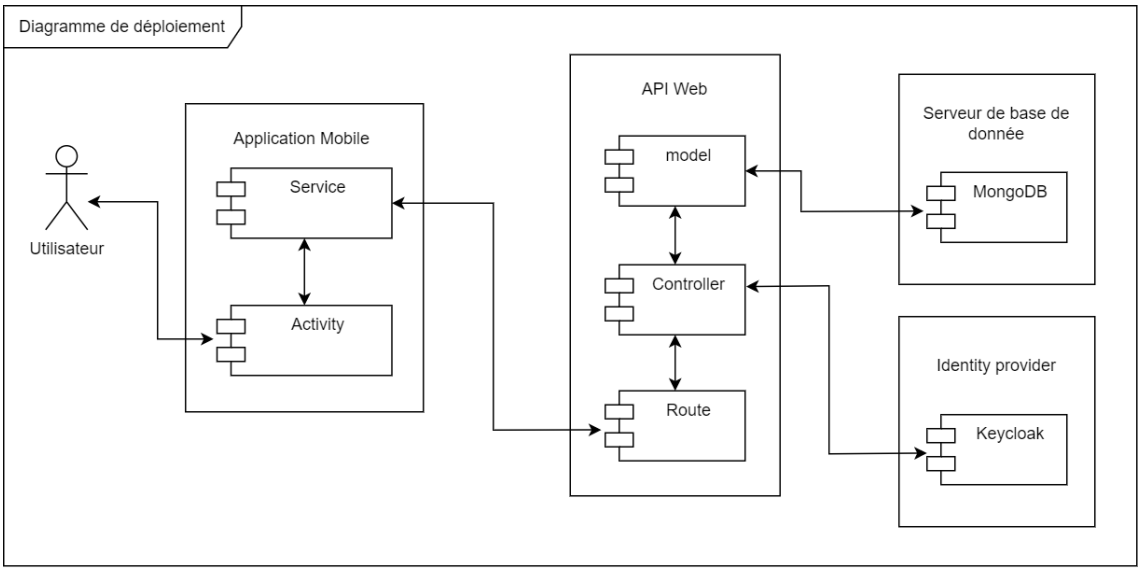


FIGURE 4.1 – Diagramme de déploiement

4.4 Travail réalisé

Dans cette rubrique nous allons présenter notre travail pour réalisé notre application en articulant sur ses différentes interfaces et fonctionnalités.

4.4.1 Continuous Integration/Continuous Delivery | CI/CD

Afin d'accélérer le processus de développement et l'intégration de notre service web nous allons choisi de créer un CI/CD pipeline en utilisant la plateforme as a service Heroku, cette dernière nous garantit une amélioration continue et un

environnement stable au niveau du test et de la livraison.

La figure 4.2 présente un échantillon des activités crée lors de développement de la solution Inetum Force de vente.

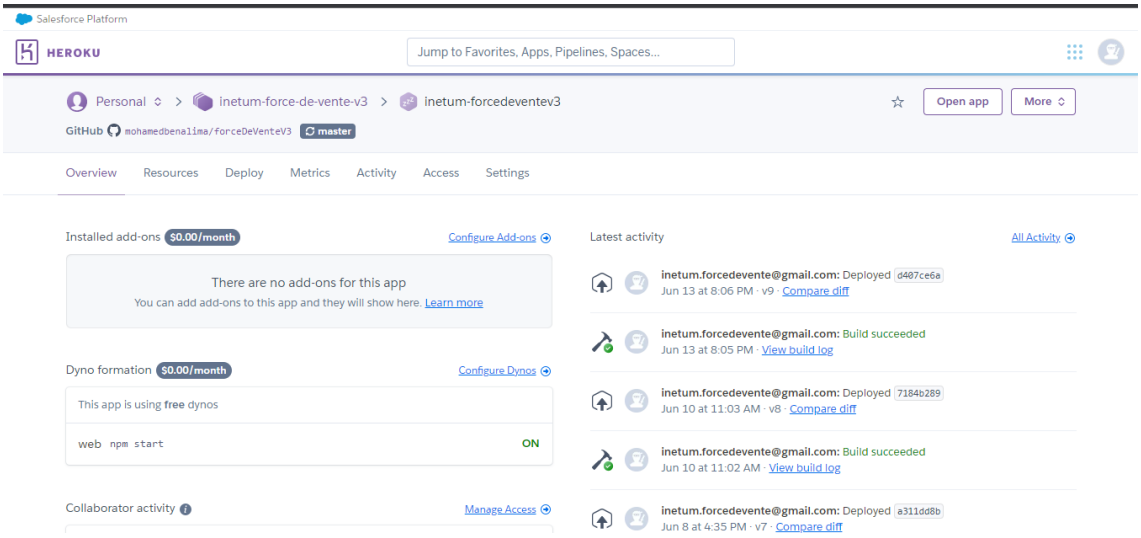


FIGURE 4.2 – plateform Heroku

4.4.2 Inetgration d'une solution SSO

Dans notre soluion Inetum force de vente, et tant que nous préparons un ensemble des produits Inetum dédié à ce concept de marketing et relation client, on à intégrer une solution SSO (Single Sign-On) avec keycloak [16] pour garantir une meilleur expérience utilisateur pour s'authentifier à toutes nos produits, et pour mieux comprendre la solution, les produits et services logiciels d'authentification unique d'entreprise (SSO) sont des gestionnaires de mots de passe avec des composants client et serveur qui connectent l'utilisateur à des applications cibles en rejouant les identifiants utilisateur. Ces identifiants sont presque toujours le nom d'utilisateur et le mot de passe, et les applications cibles n'ont pas besoin d'être modifiées pour fonctionner avec le système SSO.

La figure 4.3 présente l'interface de la solution SSO Keycloak.

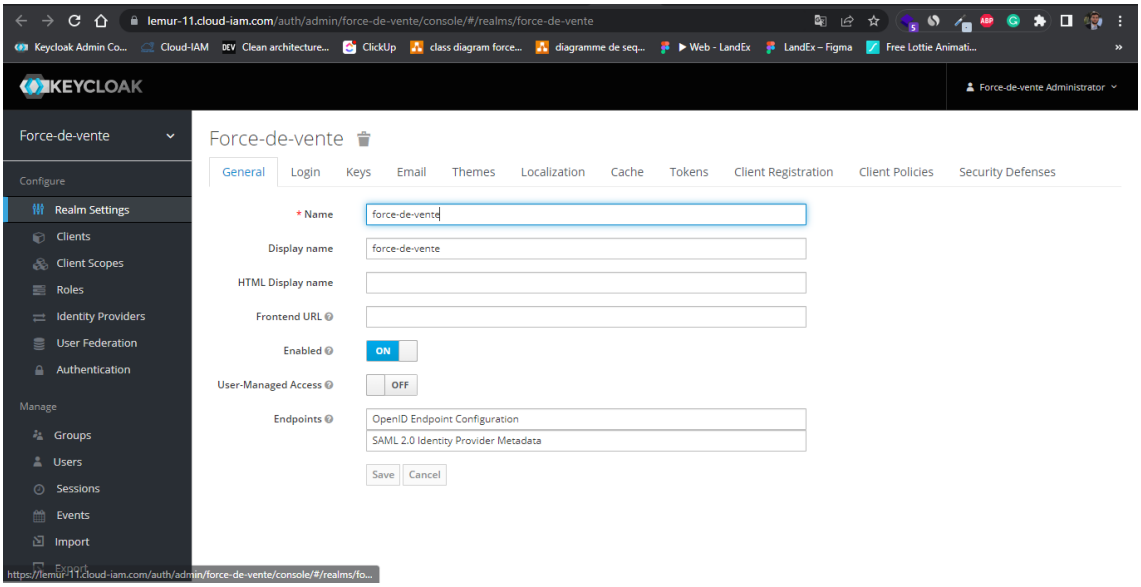


FIGURE 4.3 – Interface keycloak

4.4.3 Maquettage

Avant de se plonger dans le développement des interfaces homme/machine il faut tout d'abord préparer des maquettes graphiques selon les besoins fonctionnels de l'application, ce que nous permet de garantir une meilleure expérience utilisateur et des belles interfaces qui respectent les règles de UI/UX, et après avoir la confirmation de l'équipe de design nous avons fini à préparer la maquette de figure 4.4.

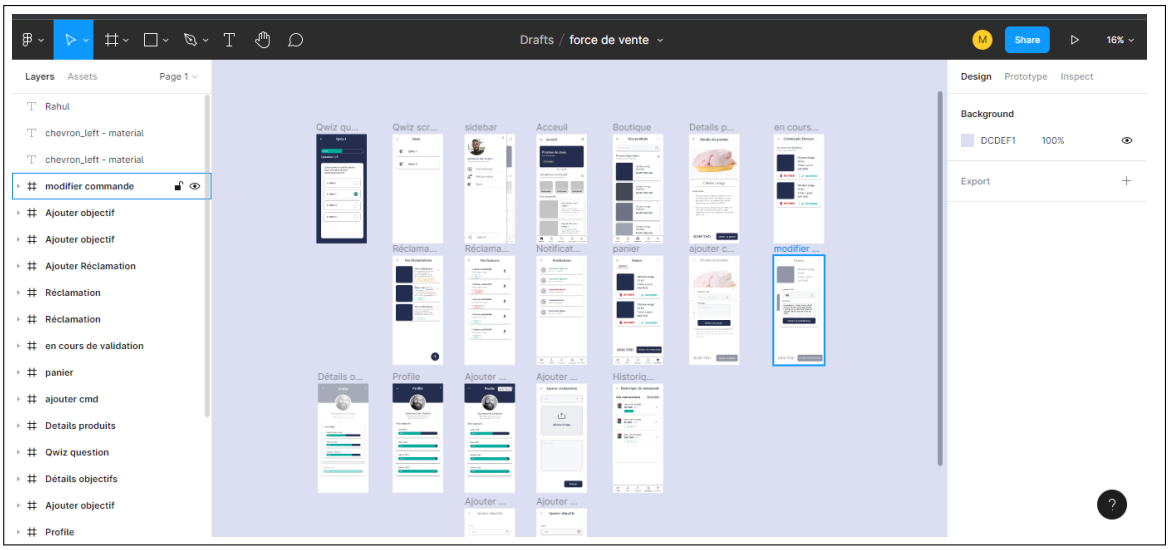


FIGURE 4.4 – Maquette graphique de l'application Inetum Force de vente

4.4.4 Interface choix de rôle

L'interface 4.5 représente la première interaction entre l'application et l'utilisateur afin de spécifier le rôle d'acteur qui souhaite se connecter.



FIGURE 4.5 – Interface choix de rôle

4.4.5 Interface Authentification

L'interface 4.6 représente le laissé passer pour un utilisateur. Pour que l'application lui donne l'accès à son compte et disposer de toutes les actions permises pour son statut après être bien authentifié. L'application vérifie aussi si le nom d'utilisateur saisi correspond à un utilisateur existant dans le fournisseur d'identité keycloak,

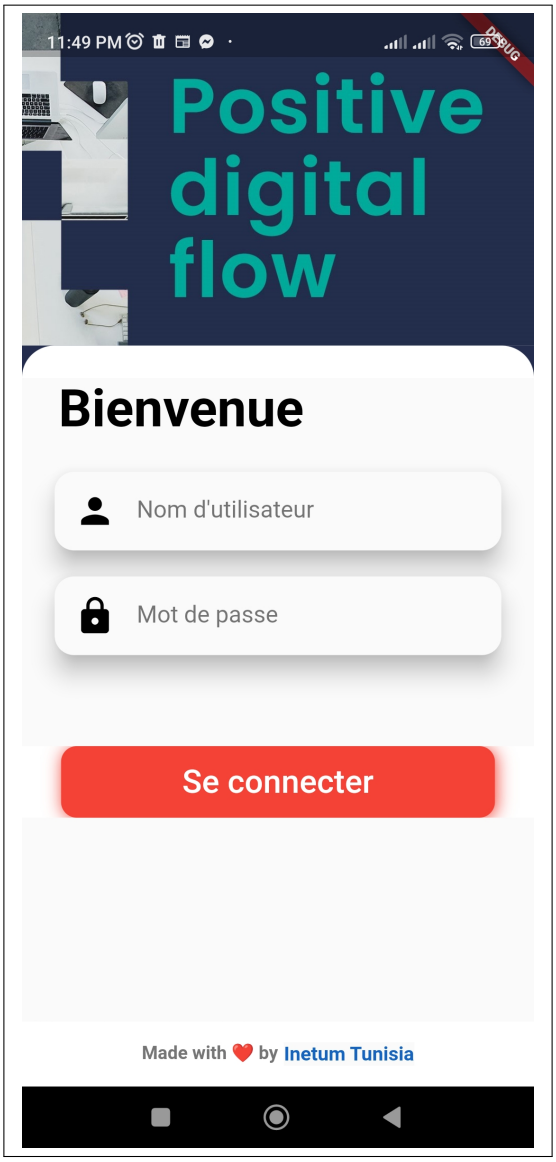


FIGURE 4.6 – Interface d'authentification

4.4.6 Interface d'accueil

L'interface 4.7 représente la page d'accueil de l'application que se lance directement après l'authentification, elle contient une barre animé qui affiche les promotions de l'entreprise, une liste horizontale qui affiche les derniers commande commander par l'utilisateur et une liste des actualités.

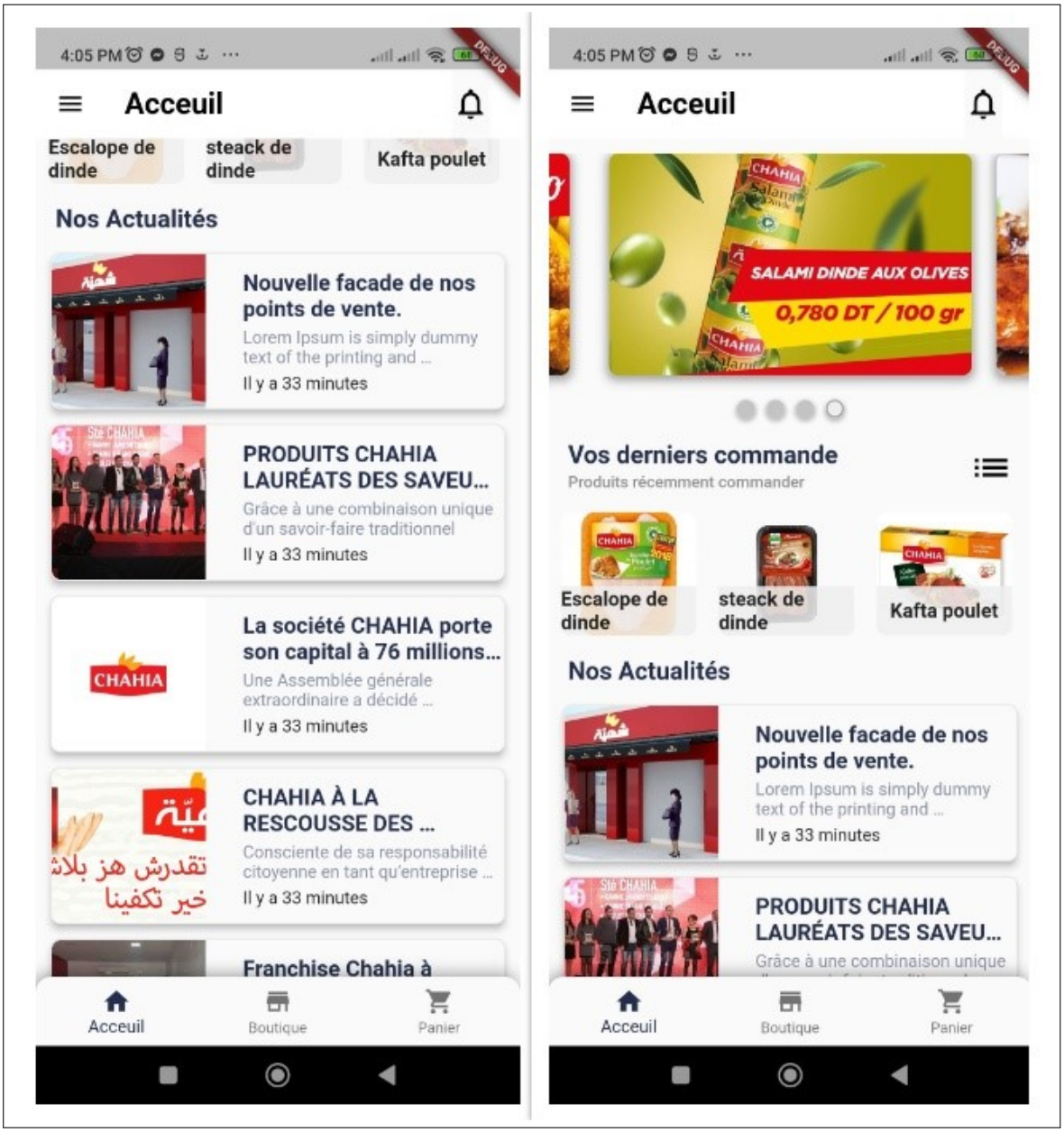


FIGURE 4.7 – Interface d'accueil

4.4.7 Interface catalogue produits

L'interface 4.8 représente le catalogue produits de l'application d'où l'agent de point de vente peut consulter les produits disponible.

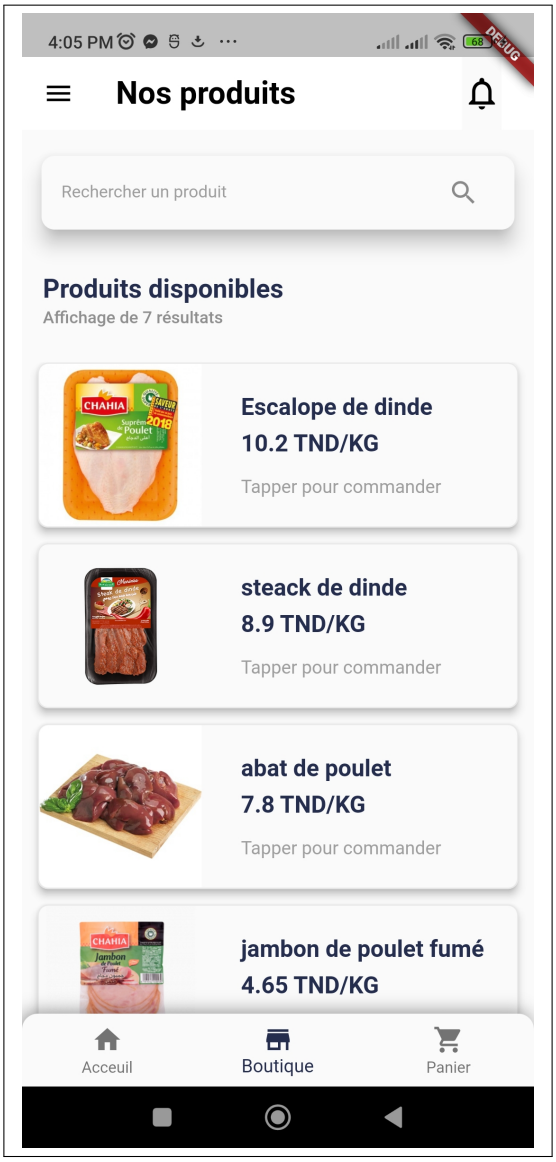


FIGURE 4.8 – Interface catalogue produits

4.4.8 Interface de détail de produit et commande

L'interface 4.9 représente les détails du produit sélectionnée par l'utilisateur avec un bouton en bas pour commander, ainsi qu'une description générale du produit : prix, label et description.

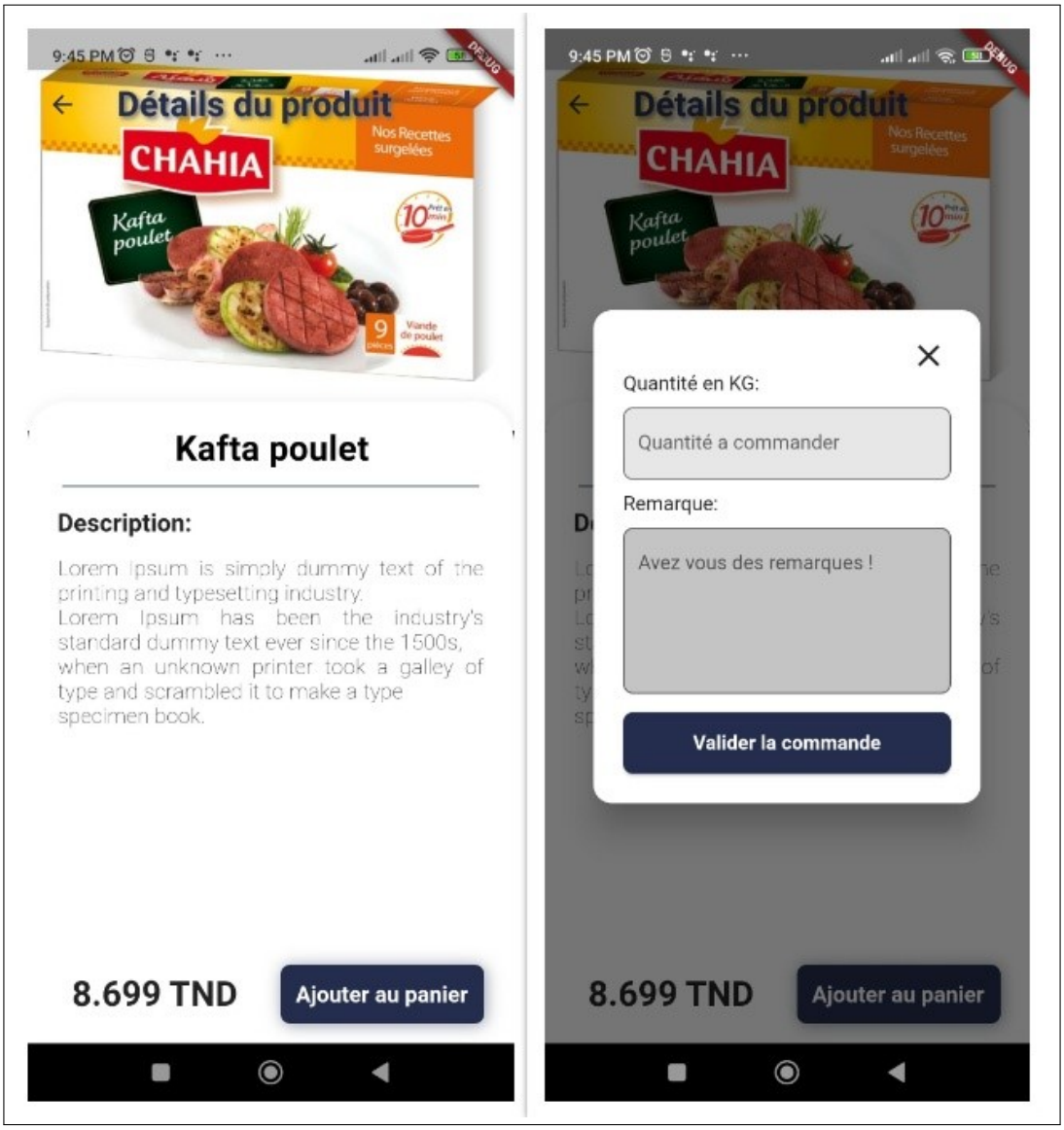


FIGURE 4.9 – Interface détails du produit

4.4.9 Interface panier

L'interface 4.10 représente le panier de l'application qui permet à l'utilisateur de collecter les produits à commander dans un seul espace, le modifier ou le supprimer avant de l'envoyer au commercial pour la validation.

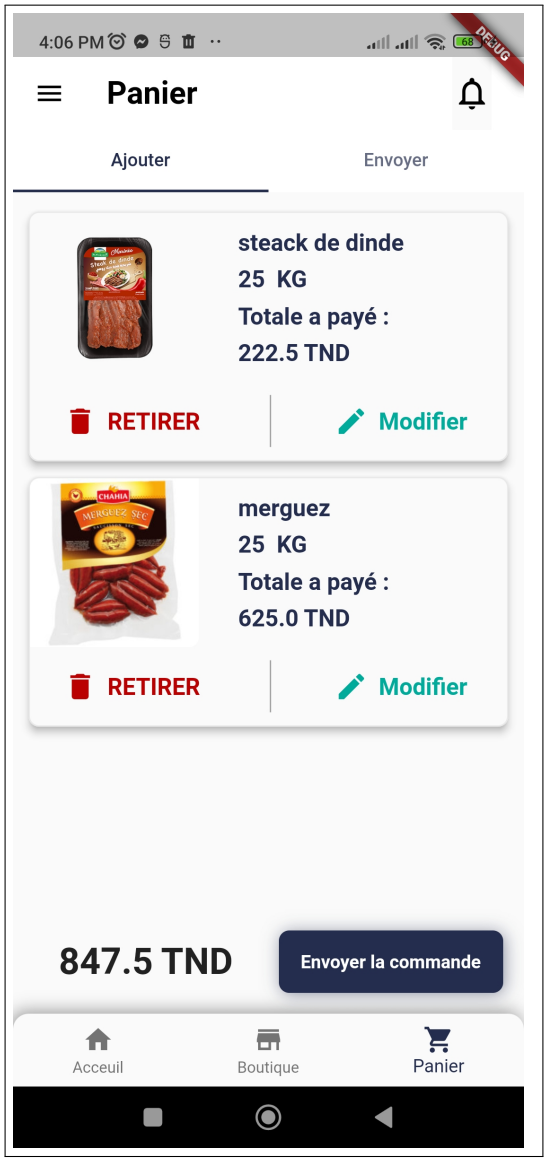


FIGURE 4.10 – Interface panier

4.4.10 Interface commande envoyer

L'interface de la figure 4.11 représente les ordres envoyés par l'agent de point de vente et lui permet de suivre son statut.

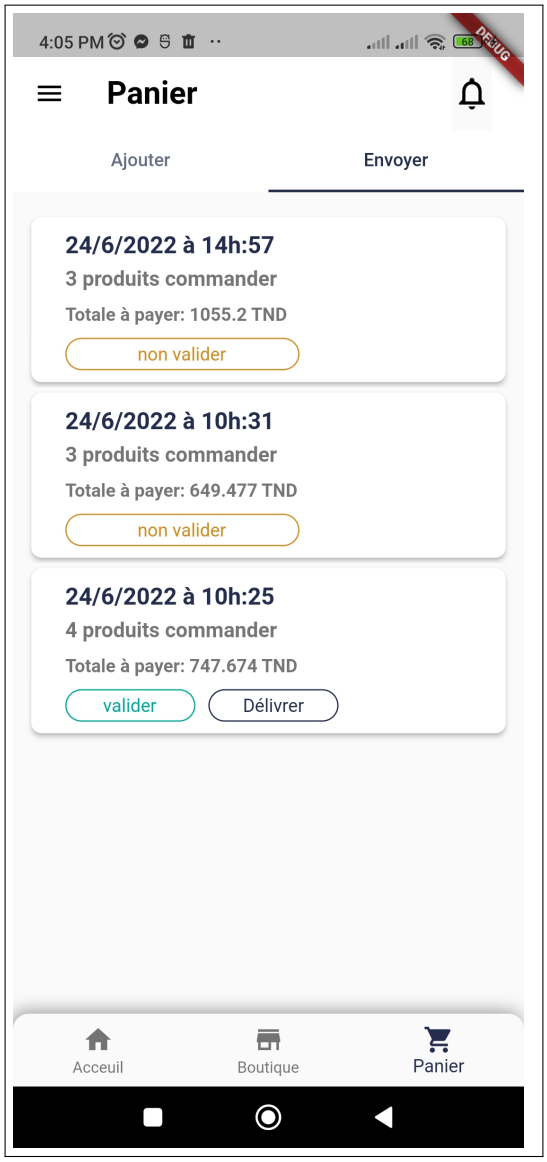


FIGURE 4.11 – Interface commande envoyée

4.4.11 Interface profile

L'interface 4.12 représente essentiellement le profile d'utilisateur et ses objectifs mensuels.

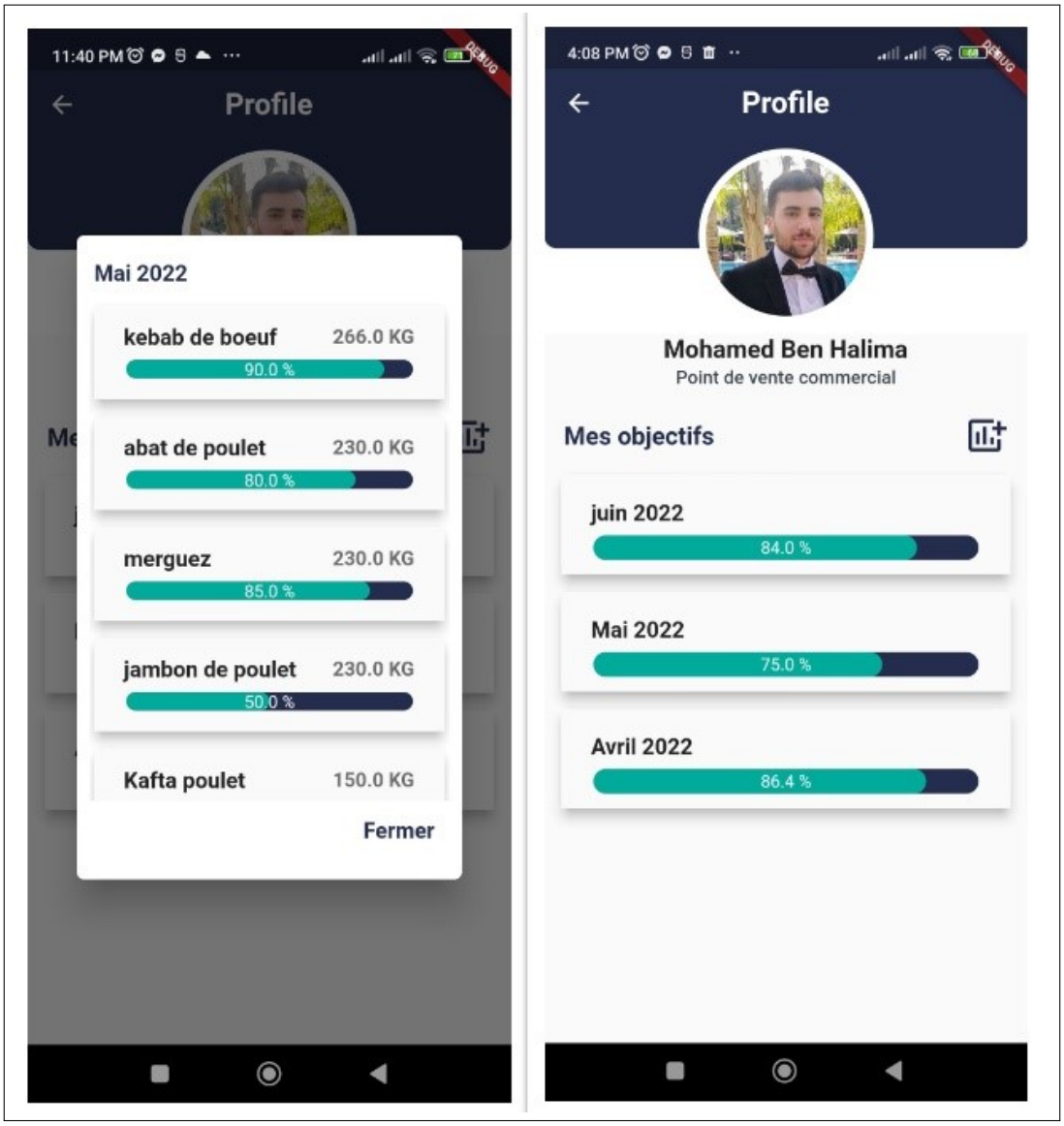


FIGURE 4.12 – Interface profile

4.4.12 Interface ajouter objectif

Les interfaces de la figure 4.13 est dédié pour l'ajout de objectifs mensuelle.

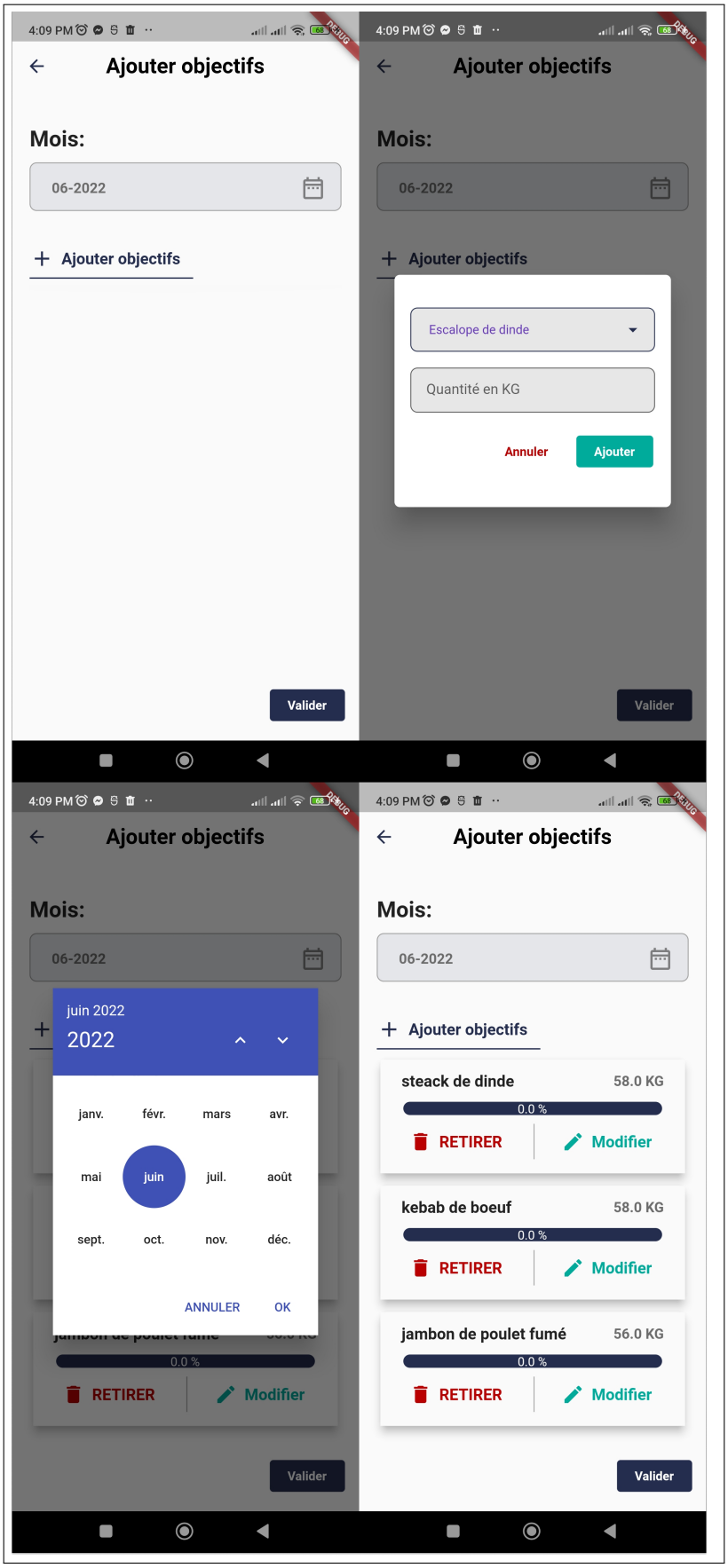


FIGURE 4.13 – Interface 'ajouter objectif'

4.4.13 Interface menu barre

L'interface de la figure 4.14 permette la commutation entre les différentes options de l'application, elle est composé par des information sur l'utilisateur en haut et des icônes d'action en bas.

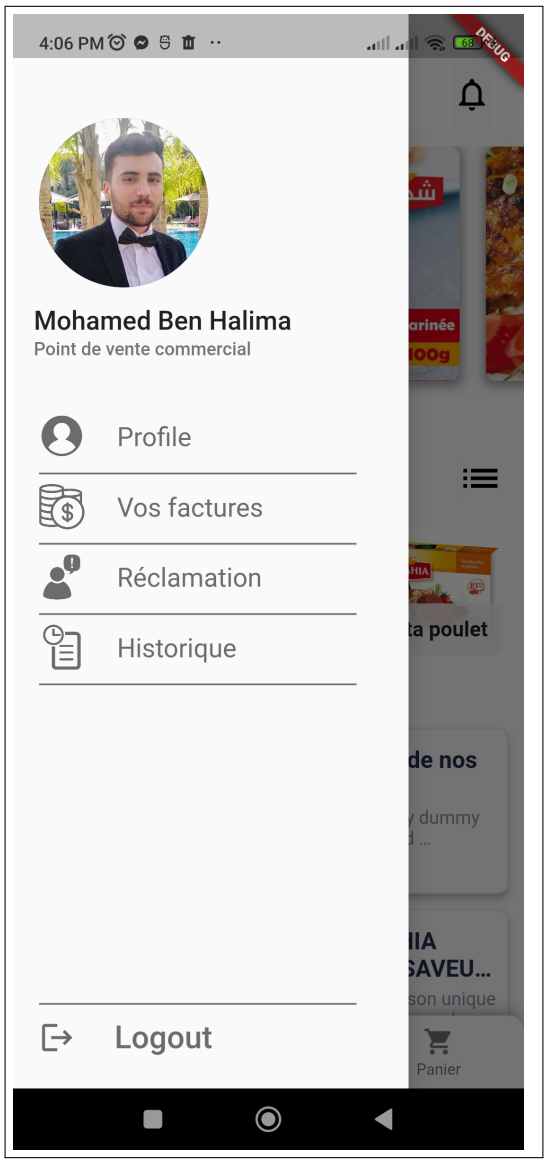


FIGURE 4.14 – Interface menu barre

4.4.14 Interface Factue

L'interface représenté par la figure 4.15 permet à l'utilisateur d'accéder à ses factures générer par le backend de la solution

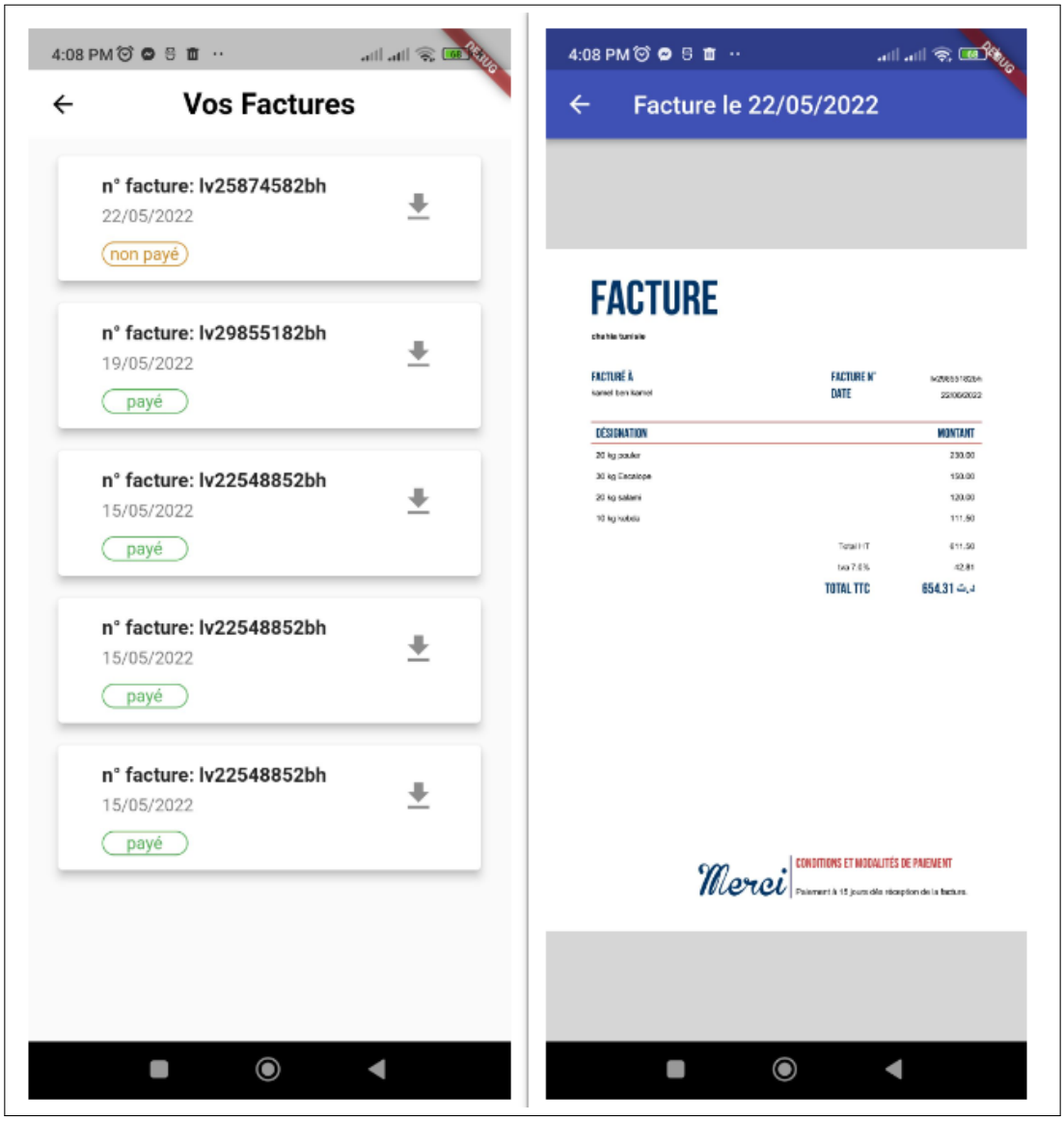


FIGURE 4.15 – Interfaces facture

4.4.15 Interface réclamation

L'interface de la figure 4.16 permette aux utilisateurs d'accéder à ses réclamations et son état de traitement avec un bouton flottante qui permet d'accéder à l'interface 'ajouter réclamation'.

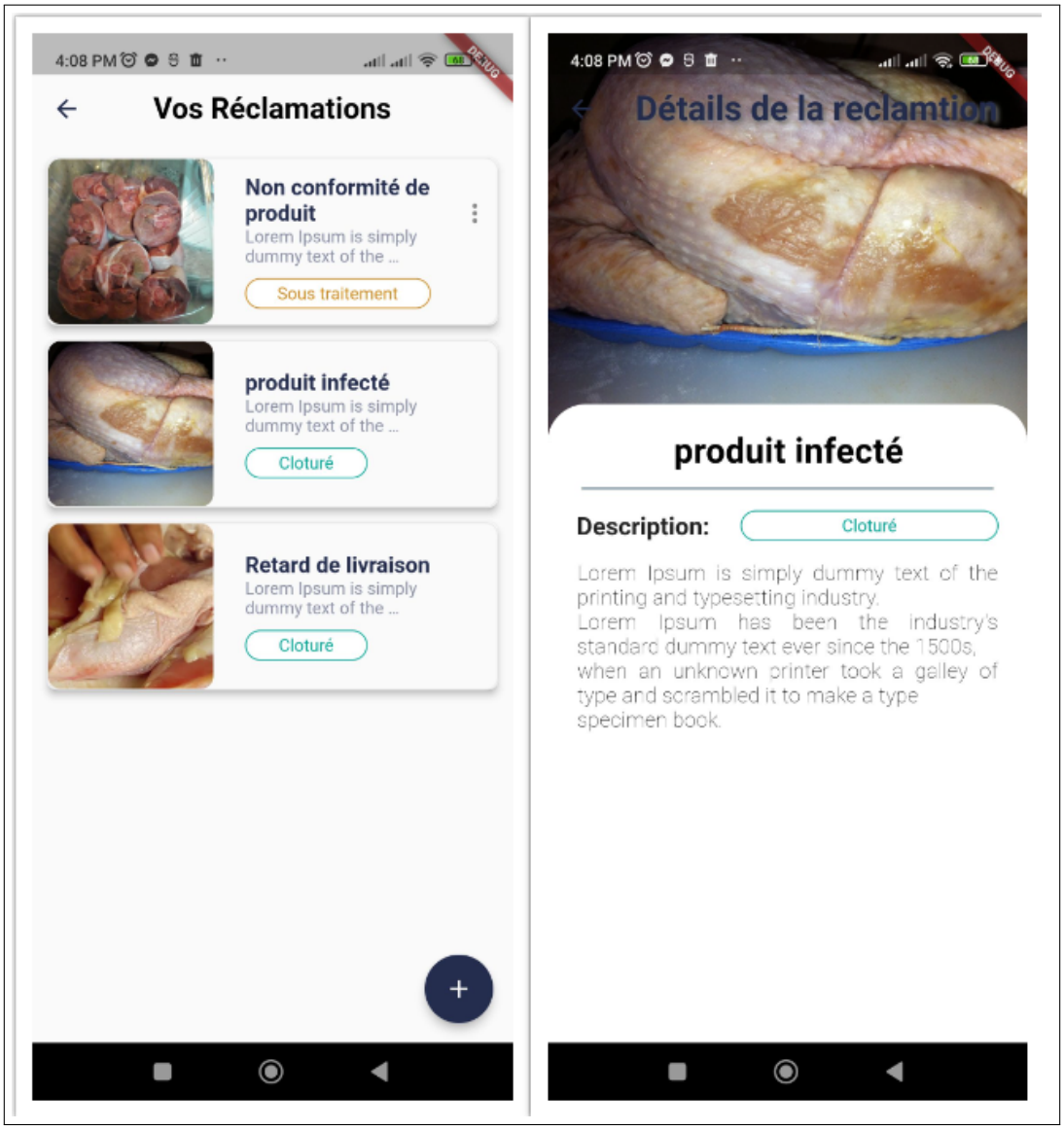


FIGURE 4.16 – Interfaces réclamation

4.4.16 Interface ajouter réclamation

L'interface de la figure 4.17 permette aux utilisateurs d'envoyer des réclamations à l'entreprise.



FIGURE 4.17 – Interfaces ajouter réclamation

4.4.17 Interface historique de commande

L'interface de la figure 4.18 permette aux utilisateur d'accéder à l'historique de commande réalisée par mois.

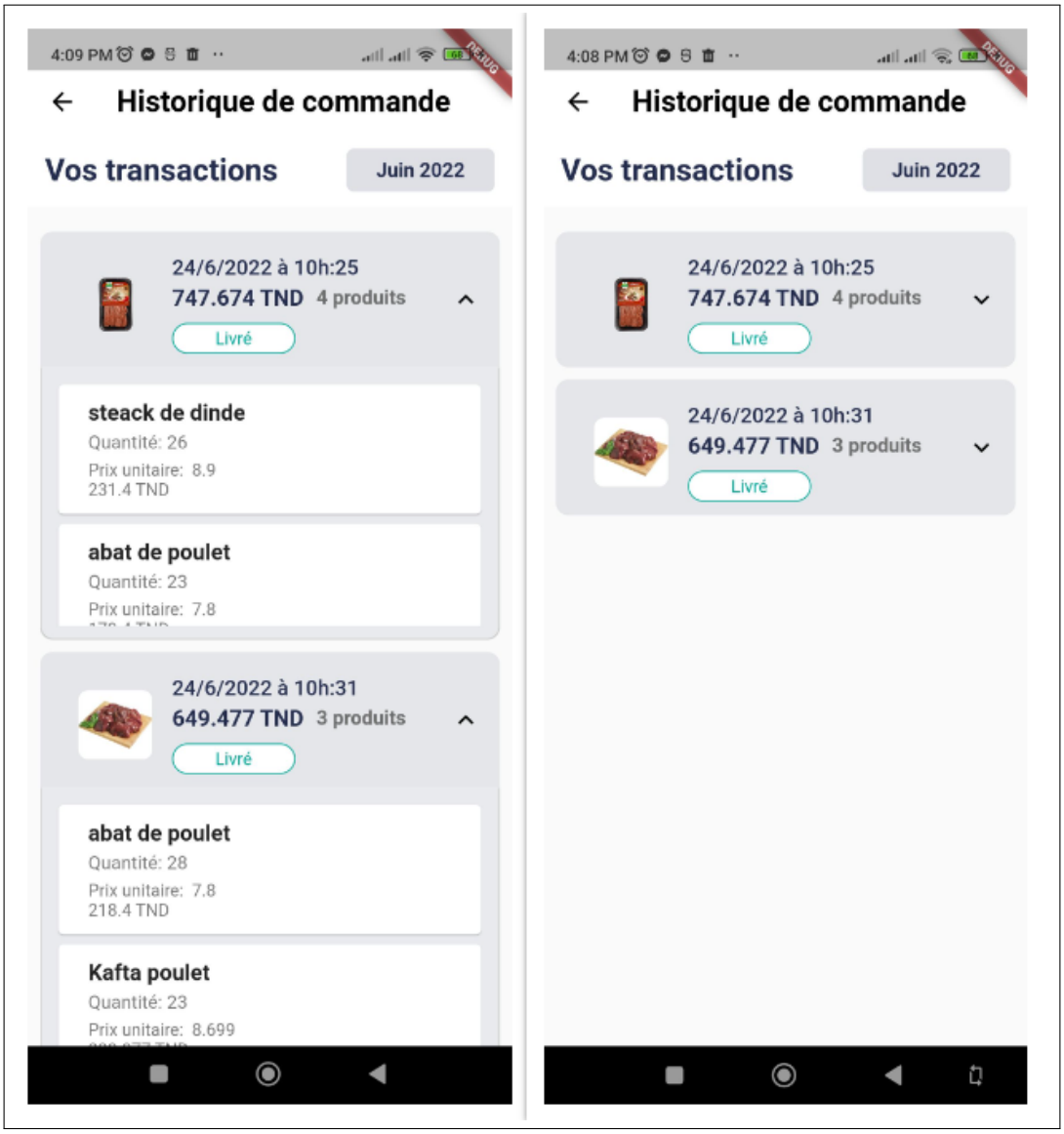


FIGURE 4.18 – Interfaces historique de commande

Conclusion

Ce dernier chapitre met en valeur notre travail réalisé en mentionnant les outils logiciels utilisés et les différentes réalisations développés pendant le stage, en articulant sur les différentes interfaces et les solutions d'intégration.

Faute du temps, pendant ce chapitre nous avons présenté que l'application dédiée à l'agent de point de vente.

Conclusion et prespectives

Ce projet était une opportunité pour mettre en place une application mobile qui sert essentiellement à digitaliser le concept de force de vente, à travers la préparation d'un ensemble des fonctionnalités qui aide les entreprises à augmenter la vente de ses produits et garantir une bonne communication avec ses clients. Cette expérience a été plus précieuse que ce à quoi je m'attendais. En plus de maîtriser le framework Flutter, la solution SSO keycloak, l'environnement d'exécution node.js et tout les technologies et les approches utilisées dans le projet, j'ai appris à gérer un environnement de travail dynamique et les traiter avec des coéquipiers et des clients d'origine nationale. Je peux avouer aussi que j'ai acquis beaucoup de compétence grâce à la culture de travail chez Inetum qu'est basée sur l'efficacité et le perfectionnisme.
Ce projet est un noyau sur lequel peut s'étendre plusieurs perspectives pour l'enrichir et le développer. En effet, le travail que nous avons réalisé est modulaire et évolutif. Il est donc possible de :

- Continuer à développer l'application commercial de la solution.
- Préparer une interface web pour l'Admin de l'application.
- Améliorer l'application dédier à l'agent de point de vente.
    - Digitaliser le processus de paiement.
    - Ajouter un module de formation continue.

Bibliographie

[1] 3tiers. https://www.techtarget.com/searchsoftwarequality/definition/3-tier-application.

[2] androidstudio. https://developer.android.com/studio.

[3] clean architecture. https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html.

[4] cleancoder. https://github.com/sdcuike/Clean-Code-Collection-Books/blob/master/The%20Clean%20Coder-A%20Code%20of%20Conduct%20for%20Professional%20Programmers.pdf.

[5] clickup. https://clickup.com/.

[6] cloudiam. https://iam.cloud-iam.com/auth/realms/cloud-iam/protocol/openid-connect/auth?client_id=cloud-iam-ui&redirect_uri=https%3A%2F%2Fapp.cloud-iam.com%2Fdeployments%3Forganization_id%3D6f0ea7cf-9e94-40d9-9321-53bc7de3578b%26page%3D1%26size%3D15%26sortField%3Dname%26sortOrder%3Dasc%26name%3D&state=dccf9669-734a-4e56-aa2c-73f26d7b9f3e&response_mode=fragment&response_type=code&scope=openid&nonce=ab496ce3-7111-4aa1-8aab-d1fde878f5bd.

[7] cloudinary. https://cloudinary.com/.

[8] dart. https://dart.dev/.

[9] drawio. https://www.diagrams.net/.

[10] expressjs. https://expressjs.com/.

[11] figma. https://www.figma.com/.

[12] flutter. https://flutter.dev/.

[13] github. https://github.com/.

[14] heroku. https://id.heroku.com/login.

[15] js. https://www.javascript.com/.

[16] kc. https://www.keycloak.org/.

[17] mongodb. https://www.mongodb.com/.

[18] mongodbatalas. https://www.mongodb.com/atlas.

[19] MVC. https://zestedesavoir.com/tutoriels/646/apprenez-a-programmer-en-java/558_java-et-la-programmation-evenementielle/2718_mieux-structurer-son-code-le-pattern-mvc/.

[20] nodejs. https://nodejs.org/.

[21] overleaf. https://www.overleaf.com.

[22] postman. https://www.postman.com/.

[23] provider. https://pub.dev/packages/provider.

[24] sellsy. https://go.sellsy.com/.

[25] vscode. https://code.visualstudio.com/.

[26] yuto. https://www.yuto.fr/solution-e-commerce-strategie-digitale/.